VectorGraphNET: Graph Attention Networks for Accurate Segmentation of Complex Technical Drawings

Andrea Carrara, S.M.ASCE[1]; Stavros Nousias, Ph.D.[2]; and André Borrmann, Ph.D.[3]

Abstract: This study introduces a new approach to extracting vector data from technical drawings in Portable Document Format (PDF) format and analyzing them semantically by employing graph attention networks. The proposed method involves converting PDF files into Scalable Vector Graphics (SVG) format and creating a feature-rich graph representation, which captures the relationships between vector entities using geometrical information. We then apply a graph attention transformer with hierarchical label definition to achieve accurate line-level semantic segmentation. Our approach is evaluated on two data sets, including the public FloorplanCAD data set, which achieves state-of-the-art results on weighted F1-score (89%), surpassing existing methods. Our vector-based method offers a scalable large-scale technical drawing analysis solution, requiring significantly less graphical processing unit (GPU) power than does current state-of-the-art techniques. Our method performs better semantic segmentation tasks, effectively extracting meaningful information from technical drawings. This enables reliable line-level classification, even for complex drawings such as architectural floorplans with overlapping structural and annotation layers, and opens up new applications, such as automated drawing processing. Additionally, our method can improve existing workflows in the Architecture, Engineering, and Construction (AEC) industry, including automated building information modeling (BIM) and construction planning. **DOI: 10.1061/JCCEE5.CPENG-6508.** *This work is made available under the terms of the Creative Commons Attribution 4.0 International license, https://creativecommons.org/licenses/by/4.0/.*

Practical Applications: Technical drawings are essential for communication in the AEC industries; however, manual analysis is often time-consuming and error prone. This study presents an efficient method for automating the segmentation and analysis of technical drawings using graph-based machine-learning techniques. Converting vector data from commonly used PDF or computer-aided design (CAD) files into feature-rich graph representations enables precise and scalable classification of drawing elements, such as walls, windows, symbols, and annotations. A key advantage of this method is its efficiency, requiring limited computational resources. Unlike traditional machine learning models that rely on computationally intensive pixel-based approaches, this framework uses lightweight graph neural networks, making it accessible for professionals without access to high-performance hardware. This efficiency allows it to process large and complex technical drawings on standard GPUs or even in cloud-based environments, ensuring broader adoption across diverse organizational scales. In practice, this approach reduces project documentation time, improves accuracy, and streamlines workflows such as BIM. It is particularly beneficial for large-scale projects with intricate technical drawings, for which traditional methods struggle to maintain precision. This research demonstrates the transformative potential of AI-driven tools to make data-driven construction and planning more efficient and accessible to a wider range of practitioners.

Introduction

The Architecture, Engineering, and Construction (AEC) industry relies heavily on technical drawings to convey complex design information and facilitate stakeholder collaboration. However, the traditional manual review and analysis of technical drawings can be time-consuming, prone to human error, and limited in scope (Borrmann et al. 2018). The digitization of technical drawings and the application of automatic content analysis can unlock new insights, enhance project outcomes, and drive innovation in the AEC sector (Moreno-García et al. 2019).

Despite the growing importance of Building Information Modeling (BIM), traditional technical drawings remain the primary mode of communication between architects/engineers and constructors and are often the sole source of geometric and semantic information for many existing buildings. The development of advanced technologies, such as computer vision and machine learning, has enabled the automation of technical drawing analysis, unlocking new opportunities for data-driven decision making and improved project delivery. Additionally, the analysis of technical drawings by human experts hinders the optimization of time and cost for an estimated 65% of companies. In contrast, adopting digitalization and BIM has been shown to significantly reduce documentation time by 75% in medium-sized construction companies through the implementation of automated document management processes, particularly in areas such as revision tracking and interdepartmental communication (Lazaro-Aleman et al. 2020).

[1]Ph.D. Candidate and Chair of Computing in Civil and Building Engineering, Technical Univ. of Munich Georg Nemetschek Institute, Technical Univ. of Munich, Arcisstraße 21, Munich 80333, Germany (corresponding author). ORCID: https://orcid.org/0009-0007-7046-1354. Email: andrea.carrara@tum.de

[2]Chair of Computing in Civil and Building Engineering, Technical Univ. of Munich Georg Nemetschek Institute, Technical Univ. of Munich, Arcisstraße 21, Munich 80333, Germany. ORCID: https://orcid.org/0000-0002-2811-235X. Email: stavros.nousias@tum.de

[3]Professor and Chair of Computing in Civil and Building Engineering, Technical Univ. of Munich Georg Nemetschek Institute, Technical Univ. of Munich, Arcisstraße 21, Munich 80333, Germany. ORCID: https://orcid.org/0000-0003-2088-7254. Email: andre.borrmann@tum.de

Note. This manuscript was submitted on October 2, 2024; approved on April 8, 2025; published online on July 25, 2025. Discussion period open until December 25, 2025; separate discussions must be submitted for individual papers. This paper is part of the ***Journal of Computing in Civil Engineering***, © ASCE, ISSN 0887-3801.

J. Comput. Civ. Eng., 2025, 39(6): 04025085

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

In the construction industry, digital documents are currently the prevailing standard for sharing plans and documents among project stakeholders (Rasmussen et al. 2019). Specifically, portable document format (PDF) is currently one of the dominant formats for sharing plans and documents. Its widespread adoption is due to its ability to preserve document formatting and layout, ensuring that complex designs and technical specifications are accurately conveyed. However, this reliance on PDF documents also introduces several challenges. For instance, PDFs can be cumbersome to work with, making it difficult to extract and manipulate data, leading to inefficiencies and errors. Moreover, the lack of standardized metadata and inconsistent naming conventions can make searching, organizing, and managing extensive collections of PDF files challenging. Additionally, the static nature of PDFs can hinder collaboration and real-time feedback because changes and revisions can be challenging to track and implement. As a result, construction projects often struggle with version control, data inconsistencies, and communication breakdowns, ultimately affecting project timelines, budgets, and overall quality (Tariq and Gardezi 2023). Conversion of PDF-based technical drawings to BIM models can be laborious, expensive, and slow, requiring specialized expertise. Automating this process could significantly accelerate BIM adoption and enhance digitization efforts in the AEC industry.

This study builds on the preliminary findings described in Carrara et al. (2024) and explores the application of modern AI techniques to make technical drawing content computationally accessible, leveraging standardized PDF formats to extract vector data and reconstruct semantic information at the line level. Approaches based on pixel images and the application of convolutional neural networks (CNN) have been presented in the past. However, they face significant limitations when applied to more complex drawings with many overlapping lines, as often found in real-world documents. In this study, we show that by employing graph neural networks (GNN) to learn the relationship between vectors and recover semantic information from drawings, the analysis of AEC technical drawings can be dramatically improved.

We build on the resolution-agnostic vector format of technical drawings, demonstrating a prominent level of precision on the line elements and facilitating the direct use of these vector representations for technical drawing analysis and semantic segmentation. Specifically, our approach extracts the geometrical data for each component and transforms it into graph features. The drawings' vectors are connected using spatially inspired rules that compute geometrical relationships for graph edge feature-based descriptors. The spatial vector descriptor prepares the data for utilization within the GNN architecture. Specifically, we design and test an efficient graph attention-based network architecture comprising two layers of graph attention transformer layers, which receive a feature-rich graph representation as input and utilize the message-passing mechanism in graphs to perform semantic segmentation.

The rest of the paper is organized as follows. The RELATED WORK section discusses the existing literature on segmentation in technical drawings, highlighting previous studies' key findings, methodologies, and limitations. The PRELIMINARIES section provides an overview of the critical concepts of Scalable Vector Graphics (SVG), which is the primary format used for drawings in this study. The METHOD section describes the research design, graph creation algorithm, analysis procedures, and classification algorithms used in this study. The EXPERIMENTAL SETUP section presents the data sets used in the case study and introduces the definition of the hierarchical labels. The RESULTS section presents the study's findings, including the experiments' results on the two different data sets. The DISCUSSION section interprets the results, highlighting the study's implications, strengths, and limitations and comparing the findings with existing research. Finally, the CONCLUSION section summarizes the study's main contributions, reiterates the key findings, and outlines potential avenues for future research.

Related Work

The automated analysis of technical drawings is a complex task due to the inherent intricacies of the data. The presence of overlapping elements and symbols, such as text, measurement lines, and design features, contributes to this complexity. Furthermore, the variability in the appearance of these elements within the drawings adds to the challenge. Initial research efforts in this domain employed a combination of rule-based analysis and traditional computer vision techniques to address these issues (Macé et al. 2010; Ahmed et al. 2011).

Subsequent studies have primarily approached this problem from a computer vision perspective, focusing on vectorization and semantic segmentation (Kalervo et al. 2019; Liu et al. 2017; Dodge et al. 2017). In this context, vectorization refers to converting a raster image composed of pixels into a resolution-independent representation consisting of geometric primitives (e.g., lines, curves, shapes).

Given the rising complexity and diversity of floorplans, recent works have increasingly turned to deep learning for the semantic segmentation of pixel images. Convolutional Neural Networks (CNNs) (Wang et al. 2021; Lv et al. 2021; Zeng et al. 2019; Surikov et al. 2020), Generative Adversarial Networks (GANs) (Zhang et al. 2020; Dong et al. 2021), and Fully Convolutional Networks (FCNs) (Yamasaki et al. 2018; Wu 2024) have all been explored to improve on heuristic-based pipelines. These deep approaches have proven beneficial in learning robust features, especially when drawings exhibit considerable variations in style, content, and symbol geometry.

Graph-Based Approaches for Technical Drawings

More recently, there has been a shift toward utilizing GNNs for element detection and segmentation in technical drawings. By representing each drawing as a graph of vector primitives (e.g., line segments, arcs, circles), GNN-based approaches can naturally capture long-range geometric relationships and adjacency patterns. Notably, Simonsen et al. (2021) proposed a method to extract floor plan graphs from computer-aided design (CAD) primitives and employed a GNN to classify graph nodes, differentiating between doors and nondoors. This GNN-based approach facilitated general room detection in office buildings and demonstrated improved performance over traditional methods (e.g., Dodge et al. 2017) and Faster R-CNN (Girshick 2015) baselines. Fan et al. (2021) introduced a large-scale FloorplanCAD data set and tackled panoptic segmentation of CAD drawings, recognizing both countable objects (furniture, windows, doors) and uncountable elements (walls). Their pipeline, referred to as PanCADNet, combined CNN and GNN modules to handle symbol spotting in complex floor plans. Although it exhibited robust performance on semantic segmentation, the reliance on large CNN detectors could lead to suboptimal accuracy for thin-line or crowded symbols.

Subgraph Detection and Multihead Attention

Zheng et al. (2022) proposed GAT-CADNet, which addresses symbol identification via a subgraph search perspective. They reformulate the symbol spotting task as an adjacency-matrix prediction problem, introducing a relative spatial encoding module to preserve geometric relationships. Their cascaded edge encoding module then extracts vertex-level attention scores to predict the final subgraph structure. This method excels at capturing both local and global

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

connectivity of primitives but remains primarily symbol-centric rather than per-line-centric in its final segmentation granularity.

Transformer-Based Symbol Spotting

Fan et al. (2022) introduced CADTransformer, which tokenizes graphical primitives into discrete tokens and employs a transformer encoder for semantic and instance predictions. By adopting neighborhood-aware self-attention, hierarchical feature aggregation, and entity position encoding, CADTransformer demonstrates strong results in fine-grained floor plan segmentation. However, its many parameters and graphical processing unit (GPU) requirements can constrain practical AEC workflows.

Point-Based Symbol Representation

Most recently, Liu et al. (2024) proposed SymPoint, which treats each graphic primitive as a locally connected "two-dimensional (2D) point" and adapts point-cloud segmentation techniques for panoptic symbol spotting. Through a point transformer backbone (Zhao et al. 2021) and a mask-based spotting head (Cheng et al. 2022), SymPoint achieves state-of-the-art performance on the FloorplanCAD data set. Key innovations include an attention-with-connection module, a contrastive connection learning scheme, and a k-nearest neighbors (KNN) interpolation mechanism for downsampled masks. Although SymPoint attains strong recognition quality, it requires long training epochs and emphasizes the instance-level assembly of primitives rather than strictly per-line classification.

Positioning Our Work

Despite the significant progress in technical drawing analysis, as previously surveyed, critical challenges in fine-grained line-level understanding remain. Notably, current approaches rely on either pixel-based backbones (e.g., CNNs or Transformers operating on rasterized patches of CAD data) or graph representations that emphasize symbol-level grouping rather than line-level semantics. In contrast, we propose a fully vector-native approach, constructing a feature-rich graph representation that incorporates complex geometric relationships among primitives. Specifically, we parse each line, arc, and curve command into multiple feature dimensions (e.g., curvature, vector direction, relative distances, intersections, collinearity checks). Each primitive becomes a node in our GNN, enriched with both node-level features (length, style attributes, vector tokens) and edge-level features (geometric distances, intersection counts, angle offsets, containment checks). By leveraging a comprehensive set of geometry descriptors, our approach explicitly encodes intricate relationships such as "neighboring lines," "overlapping arcs," and "common endpoints," thereby tackling a major limitation of purely subgraph-based or bounding-box–based methods. Furthermore, we aim to segment each line individually into fine-grained classes. This strategy diverges from subgraph detection approaches that yield broader, instance-level clusters (e.g., walls, windows) without detailing the semantic role of each individual vector. In settings in which lines can overlap (such as measurement lines crossing dimension arrows, structural lines behind text baselines), this per-line granularity proves indispensable for tasks such as automated dimension extraction, symbol classification, and domain-specific content filtering. Thus, our GNN design departs from typical symbol-spotting pipelines in the following several key ways. **Feature-Rich Graph Construction**: We automatically translate PDF-based or SVG-based drawing commands into a homogeneous set of graphical primitives and define both node and edge features that capture curvature, intersections, spatial adjacency, continuity, style attributes (color, stroke width), and approximate containment checks. This yields a significantly more complex geometric descriptor than that used in GAT-CADNet, CADTransformer, or SymPoint, which typically rely on smaller sets of tokens or adjacency definitions. **Hierarchical Classification with Line-Level Focus**: Unlike prior approaches that detect symbol bounding boxes or subgraphs, our architecture classifies each line directly into a multilevel label set (e.g., broad categorization into "annotation" versus "architecture," then a more detailed class such as "dimension line," "hatching line," or "bearing wall outline"). This hierarchical labeling scheme specifically addresses the class imbalance and oversegmentation issues plaguing purely instance-based approaches. **Lightweight, Resource-Efficient GNN**: Although existing methods often incorporate heavy CNN-Transformer backbones, we demonstrate that a carefully designed GNN, fed with rich geometric descriptors, can achieve state-of-the-art or near-state-of-the-art line-level results while requiring fewer GPU resources. Indeed, our experiments confirm that offloading computationally expensive processes (e.g., intersection checks, geometry expansions) to a computer processing unit (CPU)-based preprocessing stage allows the GNN itself to remain lightweight—in both model size and GPU memory usage. **Robust to Overlapping or Ambiguous Geometry:** By building explicit graph edges for lines that touch, intersect, or share close endpoints, we capture subtle geometric relationships that might be lost in coarse bounding boxes or subgraph groupings. This also improves accuracy in cluttered technical drawings in which line overlaps can lead to frequent confusion in pixel-based or purely adjacency–matrix–based symbol spotting. Unlike previous methods, such as GAT-CADNet or CADTransformer, that primarily focus on adjacency relationships, our approach explicitly models and compares geometries at the edge level. This ensures that nuanced interactions, such as intersections or shared endpoints, are accurately represented, addressing challenges that other methods often overlook. Taken together, these design choices differentiate our method from existing GNN-based solutions (e.g., GAT-CADNet, PanCADNet, CADTransformer, SymPoint) by thoroughly exploiting the full complexity of CAD geometry at the line level, thereby delivering a finer-grained yet computationally tractable approach to the automated analysis of technical drawings.

Preliminaries

Raster and Vector Representations in Technical Drawings

Digital representations of technical drawings can be classified into two primary categories: raster images and vector graphics. Raster images, such as those encoded in JPEG or PNG formats, are composed of a grid of pixels. Consequently, when subjected to transformations such as scaling or zooming, raster images suffer from resolution-dependent degradation, leading to a loss of fidelity and geometric precision. This limitation is particularly important in the context of scanned technical drawings, where essential geometric properties, such as line connectivity, angular relationships, and object boundaries, are implicitly encoded in pixel intensity distributions rather than being explicitly defined. The absence of topological and parametric data in rasterized representations imposes constraints on computational analysis because operations such as edge detection, segmentation, and shape recognition must rely on heuristic or learning-based approximations.

In contrast, vector graphics—exemplified by formats such as SVG and CAD files—describe geometric entities using mathematically defined primitives, including parametric curves, polylines, arcs, and polygonal surfaces. This resolution-independent representation

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

ensures that shapes retain mathematical continuity and precision regardless of transformations such as zooming or scaling. Unlike raster representations, vector-based models explicitly encode critical geometric attributes, including endpoint coordinates, curvature, adjacency relationships, and hierarchical containment structures. In the context of technical drawings, having direct access to vectors simplifies tasks such as line-level classification because the algorithm can rely on explicit geometric data (e.g., lengths, angles, and curvature) rather than approximating these features from pixel patterns.

It is crucial to recognize that for more than three decades, the overwhelming majority of technical drawings have been created using CAD systems, which usually export their results in vector-based PDF format. This underscores the practical significance of the proposed approach, which leverages the available vector-based information.

Scalable Vector Graphics

The Portable Document Format (PDF), standardized as ISO 32000 (ISO 2008), is a file format for presenting documents in a platform-independent manner. Initially developed by Adobe in 1992, PDFs encapsulate a complete description of a fixed-layout flat document, including text, fonts, vector graphics, and raster images. However, the inherent design of PDFs as a page-description language focused on describing the layout and appearance of a page rather than the underlying vector data, which poses a significant challenge for direct access to vector information.

This limitation results from PDFs being intended as a final-form format, optimized for printing or viewing rather than for editing or manipulating the underlying data. As a result, the vector graphics, including paths, shapes, and text, are often encoded in a manner that complicates the extraction of the original vector data.

The documents in a PDF are converted to SVG to allow vector extraction and manipulation. In contrast to other vector formats, such as Encapsulated PostScript (EPS), the conversion to SVG offers greater accessibility and editability. EPS files are often encoded in a binary code that requires specialized software to read and edit, limiting their flexibility and accessibility. The proprietary nature of EPS also makes it challenging to create software that can accurately parse and manipulate EPS files. In contrast, SVG is an open, text-based standard that can be easily edited using various software tools and programming languages.

The SVG language is an Extensible Markup Language (XML)-based vector image format for defining 2D graphics, as defined in the SVG 2 specification (w3S). The SVG format is maintained and standardized by the World Wide Web Consortium (W3C SVG Working Group 2023), an international community that develops and maintains web standards. This format allows for the creation and modification of complex graphical objects, and its open-source representation provides total access to the geometrical data, often not possible with vector data formats from proprietary computer-aided drawing tools.

In the context of SVG, a path is the fundamental building block for creating shapes. The path consists of a sequence of text-based commands rendered into geometrical shapes according to the format definition. The core concept underlying path commands is the definition of the current point, which is modified to create a shape by tracing a relative geometric line from the current position. The commands used in this study include those that define lines, curves, and movements within the SVG path syntax. Table 1 lists the commands used in this study. Although the SVG standard supports more complex commands, such as Circle and Rectangle, which are built on the path command, we convert them to the basic representation to ensure consistency in representation. Different SVG exports can yield varying representations, such as a rectangle being exported as a rectangle command in one case and as a path of four closing lines in another.

The conversion of basic shapes to path commands, as illustrated in Fig. 1, provides detailed insights into the underlying structure of SVG graphics. For instance, the representation of a rectangle demonstrates how a simple geometric shape can be decomposed into a sequence of path commands. The original rectangle, defined by the <rect> command with attributes width, height, x, and y, is equivalent to the path command <path> with the d attribute set to "m 0,0 v 1 h 1 v −1 z." This path command sequence can be broken down into individual components: the m 0,0 command moves to the starting point (0, 0), the v 1 command draws a vertical line to the point (0, 1), the h 1 command draws a horizontal line to the point (1, 1), the v −1 command draws a vertical line to the point (1, 0), and the Z command closes the path. This decomposition is the core principle underneath the tensor representation, where each decomposed path element is tokenized and used for structured input.

Method

Our data preprocessing and graph construction draw inspiration from prior graph-based approaches for CAD drawings (Zheng et al. 2022; Fan et al. 2021) yet introduce additional techniques to capture finer geometric relationships and ensure robust handling of overlapping vector elements. In contrast to existing pipelines that primarily focus on symbol grouping at the instance level [e.g., GAT-CADNet (Zheng et al. 2022) or PanCADNet (Fan et al. 2021)], our proposed framework constructs a feature-rich line-level graph of the entire drawing. Specifically, we partition each SVG

Table 1. Path commands in SVG used in research

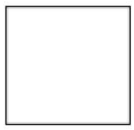
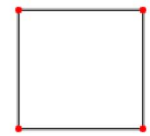
Command	Meaning	Number of parameters
M (moveto)	Move the current point to a new location.	2 (x, y)
L (lineto)	Draw a straight line from the current point to a new location.	2 (x, y)
H (horizontal lineto)	Draw a horizontal line from the current point to a new x-coordinate.	1 (x)
V (vertical lineto)	Draw a vertical line from the current point to a new y-coordinate.	1 (y)
C (curveto)	Draw a cubic Bézier curve from the current point to a new location.	6 ($x1$, $y1$, $x2$, $y2$, x, y)
S (smooth curveto)	Draw a cubic Bézier curve from the current point to a new location, using the previous control point as the first control point.	4 ($x2$, $y2$, x, y)
Q (quadratic curveto)	Draw a quadratic Bézier curve from the current point to a new location.	4 ($x1$, $y1$, x, y)
T (smooth quadratic curveto)	Draw a quadratic Bézier curve from the current point to a new location, using the previous control point as the first control point.	2 (x, y)
A (elliptical arc)	Draw an elliptical arc from the current point to a new location.	7 (rx, ry, x-axis-rotation, large-arc-flag, sweep-flag, x, y)
Z (closepath)	Close the current path by connecting to the last moveto command.	0

J. Comput. Civ. Eng., 2025, 39(6): 04025085

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

Pic	Basic Shape	Path Conversion	Pic2

	`<line`
	`style="fill:none;stroke:#000000"`
	`x1="0" y1="0"`
	`x2="1" y="1"`
	`/>`
		`<path`
		`style="fill:none;stroke:#000000"`
		`d="m 0,0 L1,1"`
		`/>`	

	`<rect`
	`style="fill:none;stroke:#000000"`
	`width="1" height="1"`
	`x="0" y="0"`
	`/>`
		`<path`
		`style="fill:none;stroke:#000000"`
		`d="m 0,0 v 1 h 1 v -1 Z"`
		`/>`
			

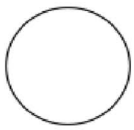
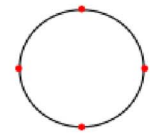
	`<circle`
	`style="fill:none;stroke:#000000"`
	`r=1`
	`cx="0" cy="1"`
	`/>`
		`<path`
		`style="fill:none;stroke:#000000"`
		`d="M 1,0`
		`A 1,1 0 0 1 0,1`
		`A 1,1 0 0 1 -1,0`
		`A 1,1 0 0 1 0,-1`
		`A 1,1 0 0 1 1,0`
		`Z"`
		`/>`
			

Fig. 1. Conversion of basic shapes in paths. Descriptions of path commands are defined in Table 1.

path into nodes and establish edge connections through standard nearest-neighbor strategies and extended geometric checks, such as shared endpoints, intersection counts, and style similarity. This level of detailed edge feature computation goes beyond earlier work, where adjacency often relied solely on bounding boxes, adjacency matrices, or simpler geometric thresholds. Moreover, we apply an efficient precomputation of curvature, area, and hierarchical token embeddings from the SVG commands, yielding a node representation that is both geometry- and style-aware. The result is a unified line-level graph that preserves complex geometric cues—an essential ingredient for accurate semantic segmentation in densely annotated technical drawings.

Therefore, the objective is to develop an automated method for segmenting line drawings at the line level utilizing solely the geometric information present in the drawing. Our ultimate goal is a fully automated process that can divide lines into distinct layers, each representing the underlying semantic meaning of the technical drawing. As illustrated in Fig. 2, our methodology comprises three primary stages. First, we perform data preparation, which involves (1) converting PDF files into the SVG format and (2) assessing and refining the data to establish a standardized format conducive to analysis. Next, we perform feature extraction and graph construction, meticulously transforming vectorized data into a rich graph that captures the geometric relationships between line entities. Finally, we define, train, and test a neural network architecture specialized in line-level semantic segmentation, leveraging the newly built graph representation to achieve high accuracy and robust performance across diverse technical drawings.

PDF-to-SVG Format Conversion and Data Structuring

We propose a PDF vector extraction framework that facilitates the systematic extraction and representation of geometric and semantic features, rendering the information computationally accessible. However, the initial SVG output often exhibits a nested, transformation-rich structure, necessitating a dedicated postprocessing step to simplify and flatten the geometries, thereby producing a transformation-invariant representation amenable to further analysis.

The process is automated through a combination of vector-specific libraries and rule-based definitions. The first step involves the conversion of PDF to SVG using CairoSVG, a 2D graphics library that provides a high-level interface for rendering SVG files. We then remove each grouping and transformation at the vector level by defining recursive functions to apply linear transformations and decoupling, thereby eliminating redundant representations.

The outcome of this process is a unique representation of the vector information, wherein each geometric entity, such as a straight wall line or an arc representing a door frame, is mapped to a distinct and unambiguous representation, eliminating the redundancy introduced by grouping and transformation operations.

Following the preprocessing stage, as illustrated in Fig. 3, each drawing is transformed into a graph representation. The graph creation process entails two key components: node feature extraction and edge feature extraction. These steps extract relevant information from the drawing, which is then encoded as node and edge attributes in the resulting graph.

Node Feature Extraction

In the graph construction process, each path within the SVG is represented as a node, to which we assign both geometrical and stylistic information encoded in the XML definition.

Style Features

We parse the XML tags to determine the presence or absence of a fill color, assigning a binary value accordingly. Additionally, we extract the stroke color and stroke width attributes.

Numerical Features

The path is converted into a vectorized format, and its features are extracted through numerical operations. Initially, we determine whether the path is closed, which is represented as a binary value.

J. Comput. Civ. Eng., 2025, 39(6): 04025085

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

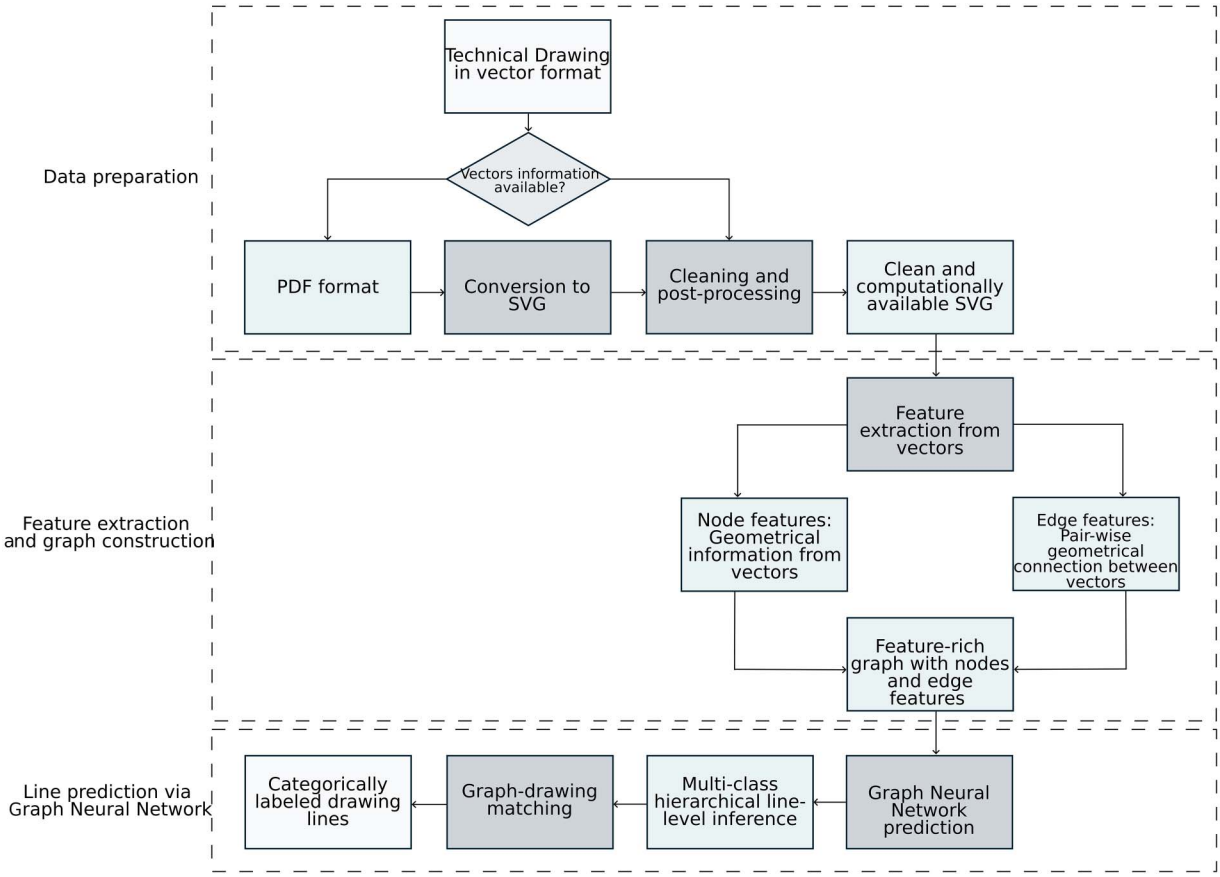


Fig. 2. Graph creation process.

When a path is closed, its area is calculated. Additionally, we compute the total path length and total number of commands across all paths. We apply a logarithmic transformation to measures susceptible to outliers, such as area and length, thereby reducing their negative influence on the model and mitigating the impact of outliers on the training process. We use a numerical method for estimating the curvature of parametric curves at multiple points along the curve. The approach involves dividing the curve into a series of intervals, with the number of intervals determined by the user-defined parameter n. The length of each interval, denoted by Δt, is calculated as the reciprocal of n

$$\Delta t = \frac{1}{n} \tag{1}$$

For each interval, we calculate the point on the curve corresponding to the current parameter value t, where t ranges from 0 to 1. The point is obtained by evaluating the parametric curve function $p(t)$ at t.

To estimate the curvature at each point, we employ a finite difference method, which approximates the second derivative of the curve. Specifically, we calculate the curvature using the following formula:

$$\kappa = \frac{2((x_0 - x_1)(y_2 - y_1) - (y_0 - y_1)(x_2 - x_1))}{\sqrt{((x_0 - x_1)^2 + (y_0 - y_1)^2)^3}} \tag{2}$$

where (x_0, y_0), (x_1, y_1), and (x_2, y_2) = points on the curve, with (x_1, y_1) = current point; and (x_0, y_0) and (x_2, y_2) = points at $t - \Delta t$ and $t + \Delta t$, respectively. We extract the median point to capture the vector's central tendency. This is achieved by sampling n equally spaced points along the path, denoted as $p_1, p_2, \ldots, p_n$ where each point p_i is characterized by its x-coordinate x_i and a y-coordinate y_i. The points are spaced evenly along the path, with a distance of $\Delta s = L/(n-1)$ between each consecutive point, where L = total path length. We then calculate the median point of these sampled points, which we denote as $\bar{p} = (\bar{x}, \bar{y})$. This median point is taken as the path's representative (x, y) point.

SVG Embedding

The final part of the node features is the tensor representation of the SVG path command following the approach presented in DeepSVG (Carlier et al. 2020). The input data are a string representation of an SVG path, which is a compact notation for describing a sequence of 2D geometric shapes described in the Scalable Vector Graphics subsection. The string is subjected to lexical analysis of the text describing the geometry in SVG format and tokenization, resulting in a sequence of tokens denoted as $\mathcal{T} = t_1, t_2, \ldots, t_n$, where each token t_i represents a command or coordinate. The token sequence is then converted into an intermediate representation, a hierarchical data structure that captures the essential information of the SVG path. This intermediate representation can be formalized as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ = set of vertices representing commands and coordinates; and $\mathcal{E}$ = set of edges representing the relationships between them.

Tensor Representation

The intermediate representation $\mathcal{I}$ is a class or data structure instance, denoted as $\mathcal{S}$, representing a group of SVG paths. Each path in the group is composed of multiple segments, where each

J. Comput. Civ. Eng., 2025, 39(6): 04025085

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

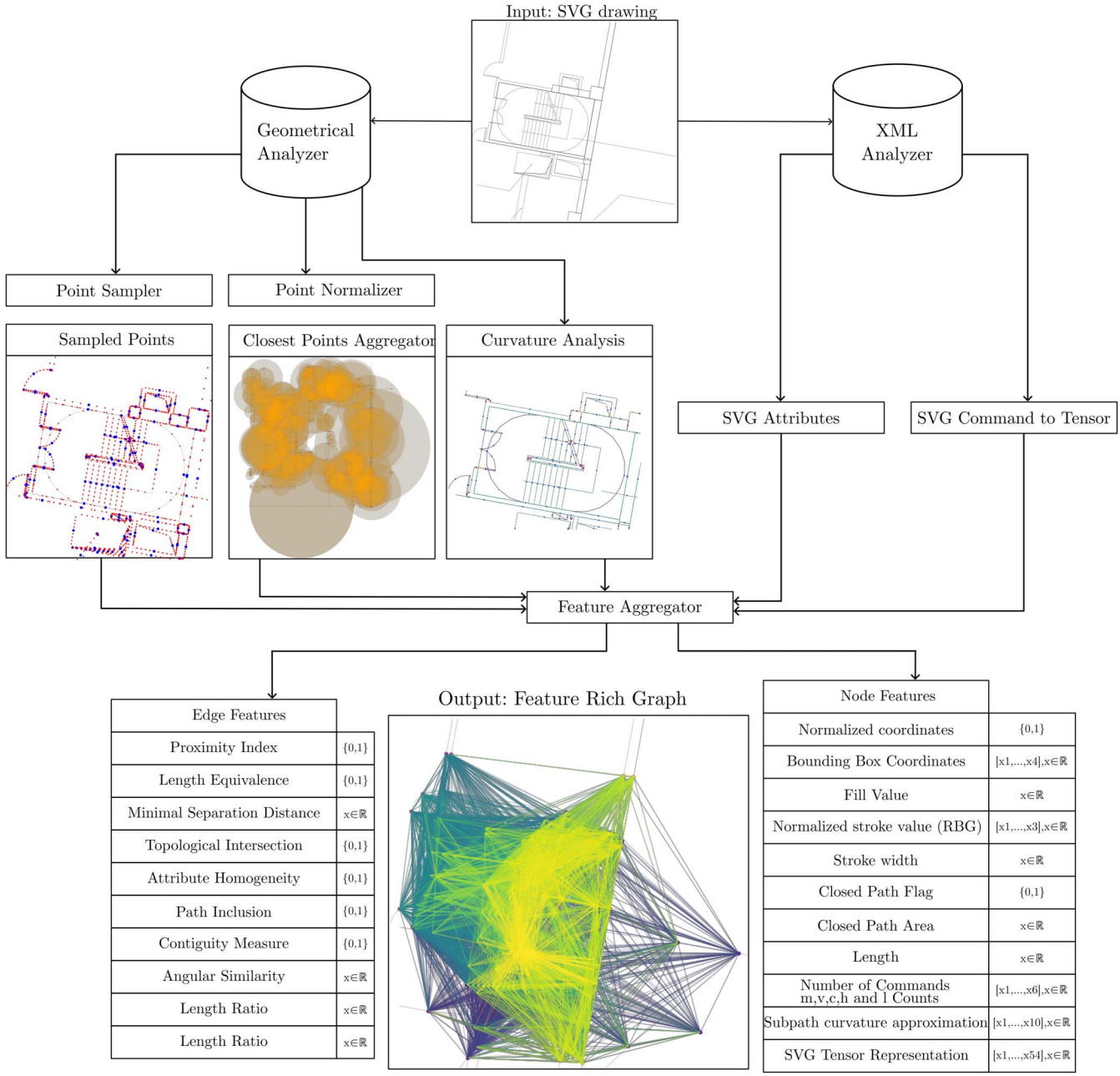


Fig. 3. Graph creation process: Geometrical and XML analysis are used to extract information, which is then aggregated into edge and node features.

segment is a sequence of points. Each segment in the path is represented by a tuple of points denoted as $\mathcal{P} = (x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)$, where each point (x_i, y_i) is a 2D coordinate. The intermediate representation $\mathcal{I}$ is then converted into a tensor representation denoted as $\mathcal{T}$, which is a numerical representation of the path data. The tensor $\mathcal{T}$ is a multidimensional array of numbers, where each dimension corresponds to a specific aspect of the path data, such as the command type, coordinates, or other parameters. The tensor structure can be formalized as a matrix $\mathcal{T} \in \mathbb{R}^{n \times m}$, where n = number of segments; and m = number of attributes per segment. Each row of the matrix represents a segment, and each column represents a specific segment attribute.

The SVG path command can comprise an arbitrary number of subcommands, enabling the creation of complex shapes with varying degrees of intricacy. To ensure consistency in the graph representation, we pad and truncate the resulting tensor to a fixed length of n. Tensors with lengths less than n are padded with vectors following the shape of the tensor filled of -1, whereas those exceeding n are truncated, potentially resulting in information loss. The choice of n is critical because selecting a value that is too small may lead to excessive information loss, whereas choosing a value that is too large may introduce redundant information for most nodes, potentially impeding or compromising the training of the network.

Edge Feature Extraction

The graph nodes encapsulate the geometric attributes of each line segment. In contrast, the edges embody the topological and semantic correlations between these line segments, capturing the spatial and relational dependencies among them.

J. Comput. Civ. Eng., 2025, 39(6): 04025085

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

To mitigate the complexity of a fully connected graph, we utilize the K-nearest neighbor (KNN) algorithm to reduce the degree of each node. Specifically, we calculate each node's top K nearest neighbors based on the average vector point obtained from the node-feature extraction process. As illustrated in Fig. 4, the choice of the hyperparameter K significantly impacts the graph's structure: a low value of K results in a sparse graph, whereas increasing K yields a more densely connected graph. To ensure the resulting graph is undirected, we remove duplicate appearances of node pairs by aggregating bidirectional edges into a single undirected edge. For instance, any pair of nodes connected in both directions are merged into a single undirected connection, effectively halving the number of edges in the graph. This simplification preserves the symmetry of the relationships between nodes while reducing the

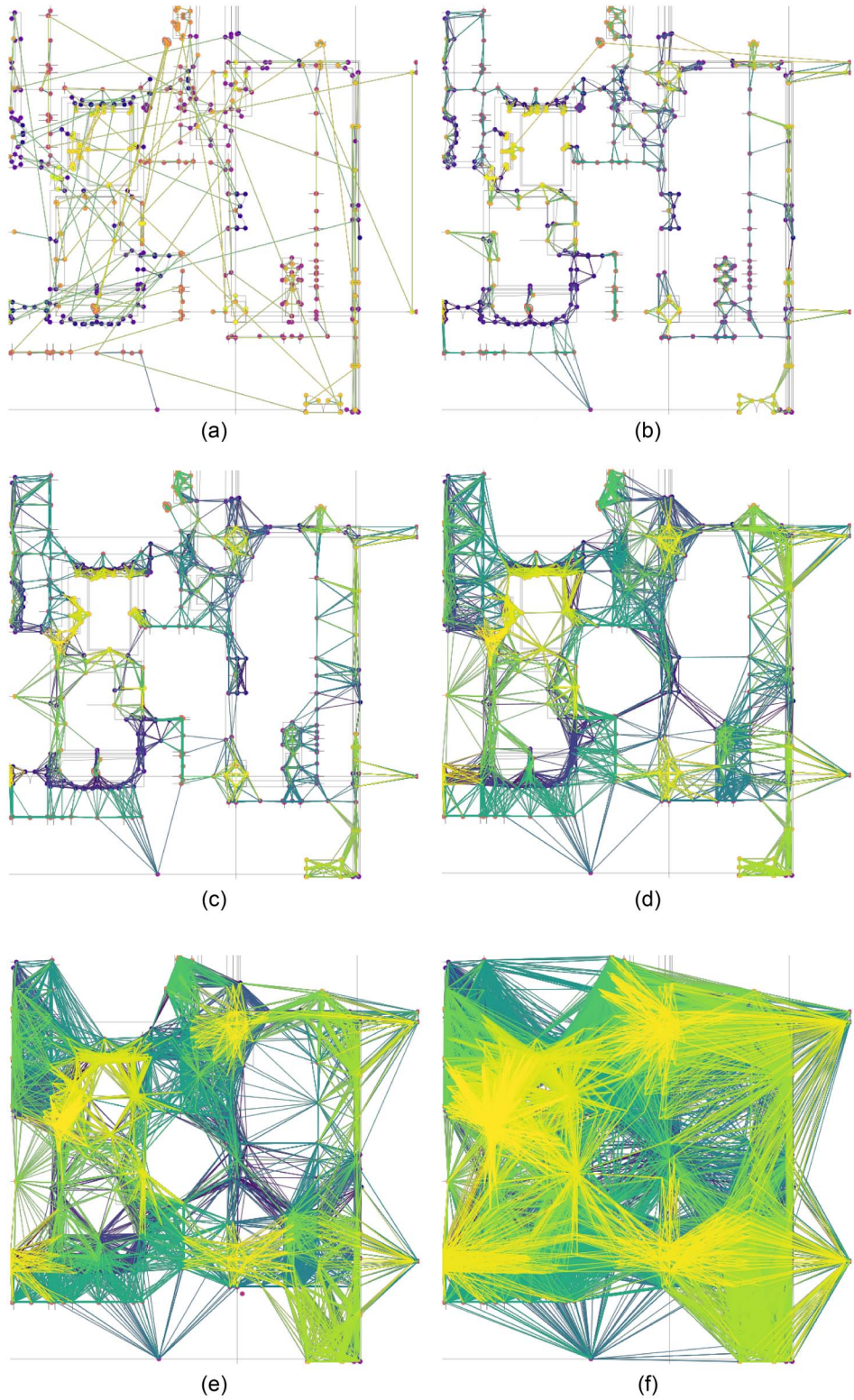


Fig. 4. Different connectivities on graphs: (a) graph with connectivity $k = 1$; (b) graph with connectivity $k = 5$; (c) graph with connectivity $k = 10$; (d) graph with connectivity $k = 20$; (e) graph with connectivity $k = 40$; and (f) graph with connectivity $k = 100$.

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

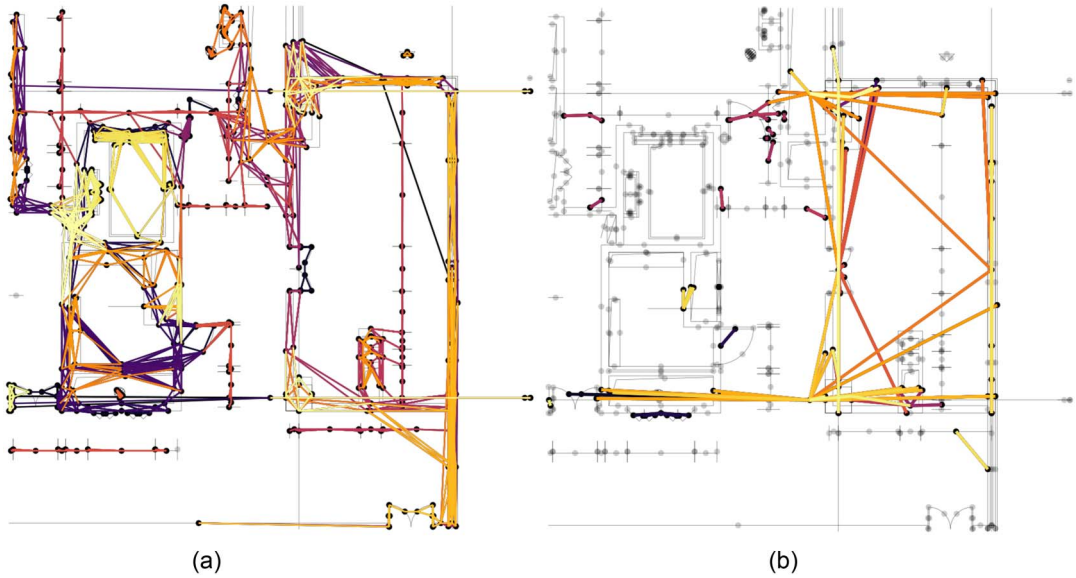


Fig. 5. Example filtering: (a) filter for contiguous lines; and (b) filter for center intersection.

overall complexity of the graph. This approach is enhanced by adding the possibility of connections between random graph nodes, which encourages the model to learn a more comprehensive representation of the symbols and semantics rather than simply relying on the local context of surrounding nodes, thereby preventing overfitting and poor generalization to new, unseen data because the model can capture long-range dependencies and relationships between nodes that may not be immediately adjacent. The KNN approach is favored over a distance-threshold cut-off method to guarantee that every node in the graph is connected to at least K other nodes, thereby ensuring a minimum level of connectivity and preventing isolated nodes.

The edge feature is a 10-dimensional vector comprising several components that capture the geometric and topological relationships between two segments. This feature enables the derivation of structural patterns and supports filtering different types of connections within the graph. For example, Fig. 5 shows the extraction of the contiguous lines and central intersection information from the graph, allowing for straightforward application of geometric filters to the drawing.

Specifically, the vector includes two Boolean values indicating whether the segments have identical lengths and whether they are selected based on the vicinity rule or random sampling. The vector also stores the two segments' logarithmic and inverse length ratios. The normalized angle between the center points of the mean points of the segments, denoted as $p1 = (x1, y1)$ and $p2 = (x2, y2)$, is calculated using the following formula:

$$\theta_{\text{norm}} = \frac{(\tan^{-1}(\frac{\Delta y}{\Delta x}) \text{mod } 360)}{360} \tag{3}$$

where $\Delta x = x_1 - x_2$ and $\Delta y = y_1 - y_2$.

To account for the potential complexity of the vector, we also incorporate the minimum point between the two geometries. This is achieved by representing each geometry using equal-distance sampling and computing the logarithm of the minimum pairwise distance matrix between the points.

In cases in which one of the two paths is a closed path, we return a binary feature of whether one path is contained within the other. Furthermore, we include the number of intersections between the two segments, which is approximated using a numerical method on the set of points sampled from the geometry, as described in the subsequent section. This approximation is necessary due to the computational expense of exact intersection calculations for numerous SVG paths. From the line segments $\mathbf{S}_1$ and $\mathbf{S}_2$ in $\mathbb{R}^2$, parameterized by $t \in [0, 1]$, are as follows:

$$\mathbf{S}_1(t) = (x_1(t), y_1(t)) = (1 - t)\mathbf{P}_1 + t\mathbf{Q}_1 \tag{4}$$

$$\mathbf{S}_2(t) = (x_2(t), y_2(t)) = (1 - t)\mathbf{P}_2 + t\mathbf{Q}_2 \tag{5}$$

where $\mathbf{P}_1, \mathbf{Q}_1, \mathbf{P}_2, \mathbf{Q}_2 \in \mathbb{R}^2$ are the endpoints of the segments. To detect intersections between $\mathbf{S}_1$ and $\mathbf{S}_2$, we sample the segments at N points each, yielding the following two sets of points:

$$\mathbf{J} = \{\mathbf{S}_1(t_i)\}_{i=0}^{N-1} \tag{6}$$

$$\mathbf{K} = \{\mathbf{S}_2(t_i)\}_{i=0}^{N-1} \tag{7}$$

We then compute the following matrices:

$$\mathbf{A}_x = \begin{bmatrix} x_1(t_0) & x_2(t_0) \\ x_1(t_1) & x_2(t_1) \\ \vdots & \vdots \\ x_1(t_{N-1}) & x_2(t_{N-1}) \end{bmatrix} \tag{8}$$

$$\mathbf{A}_y = \begin{bmatrix} y_1(t_0) & y_2(t_0) \\ y_1(t_1) & y_2(t_1) \\ \vdots & \vdots \\ y_1(t_{N-1}) & y_2(t_{N-1}) \end{bmatrix} \tag{9}$$

The intersection points are then detected by solving the following system of linear equations:

J. Comput. Civ. Eng., 2025, 39(6): 04025085

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

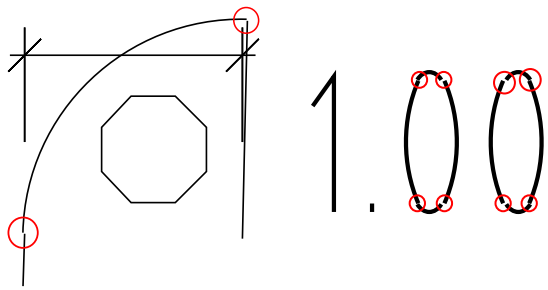


Fig. 6. Errors in representation.

$$[\mathbf{A}_x \quad -\mathbf{A}_y]\begin{bmatrix} t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} x_1(t_0) - x_2(t_0) \\ y_1(t_0) - y_2(t_0) \\ \vdots \\ x_1(t_{N-1}) - x_2(t_{N-1}) \\ y_1(t_{N-1}) - y_2(t_{N-1}) \end{bmatrix} \tag{10}$$

where $t_1, t_2 \in [0, 1]$ are the parameters of the intersection point. The solution to this system yields the intersection points, which are then filtered using the following conditions:

$$\begin{aligned} &\det(\mathbf{A}_x, \mathbf{A}_y) \neq 0 \quad \text{non-parallel curves} \\ &\det(\mathbf{A}_x, \mathbf{A}_y) \triangleq (x_1(t_0) - x_2(t_0)) \quad \text{same sign on x-axis} \\ &\det(\mathbf{A}_x, \mathbf{A}_y) \triangleq (y_1(t_0) - y_2(t_0)) \quad \text{same sign on y-axis} \\ &|\det(\mathbf{A}_x, \mathbf{A}_y)| \geq |x_1(t_0) - x_2(t_0)| \quad \text{x-axis distance} \\ &|\det(\mathbf{A}_x, \mathbf{A}_y)| \geq |y_1(t_0) - y_2(t_0)| \quad \text{y-axis distance} \end{aligned} \tag{11}$$

where $a \triangleq b$ if and only if a and b have the same sign.

A binary flag is incorporated into the feature set when two segments exhibit identical SVG style descriptors, thereby facilitating the identification of stylistically similar elements. Furthermore, the contiguity of the segments is evaluated by comparing their extreme points with a tolerance threshold to account for minor discrepancies arising from potential errors in converting SVG geometries from disparate formats.

Fig. 6 illustrates examples of such uncertainties, which predominantly occur in text that is not encoded as text in the document but rather converted into geometrical vectors. This discrepancy is noteworthy because it may not be apparent when examining the raster version of the file but significantly impacts the graph representation. Consequently, correcting these errors is essential to ensure the accuracy and reliability of the graph representation.

Graph Neural Network

In complex technical drawings, line segments may overlap, share endpoints, or be spatially distant yet semantically related (e.g., repeated symbols and collinear edges).To account for all such possibilities, we intentionally incorporate into our graph construction abundant edge connections, some of which may have minimal relevance. A Graph Attention Transformer is well-suited for this setup: its multihead attention mechanism can learn to down-weight irrelevant edges while emphasizing crucial relationships. By dynamically weighting both node- and edge-based features, it preserves fine-grained geometric details and handles long-range dependencies, ultimately enabling robust, per-line semantic classification.

To further exploit these complex relationships, we propose a neural network architecture incorporating attention mechanisms to selectively focus on the most relevant neighbors, as illustrated in Fig. 7. This approach enables the network to learn a weighted representation of the graph structure, where the importance of each neighbor is adaptively determined. Our model architecture is based on this principle and consists of a sequence of two Graph Attention v2 layers (Brody et al. 2021) interspersed with a ReLU activation

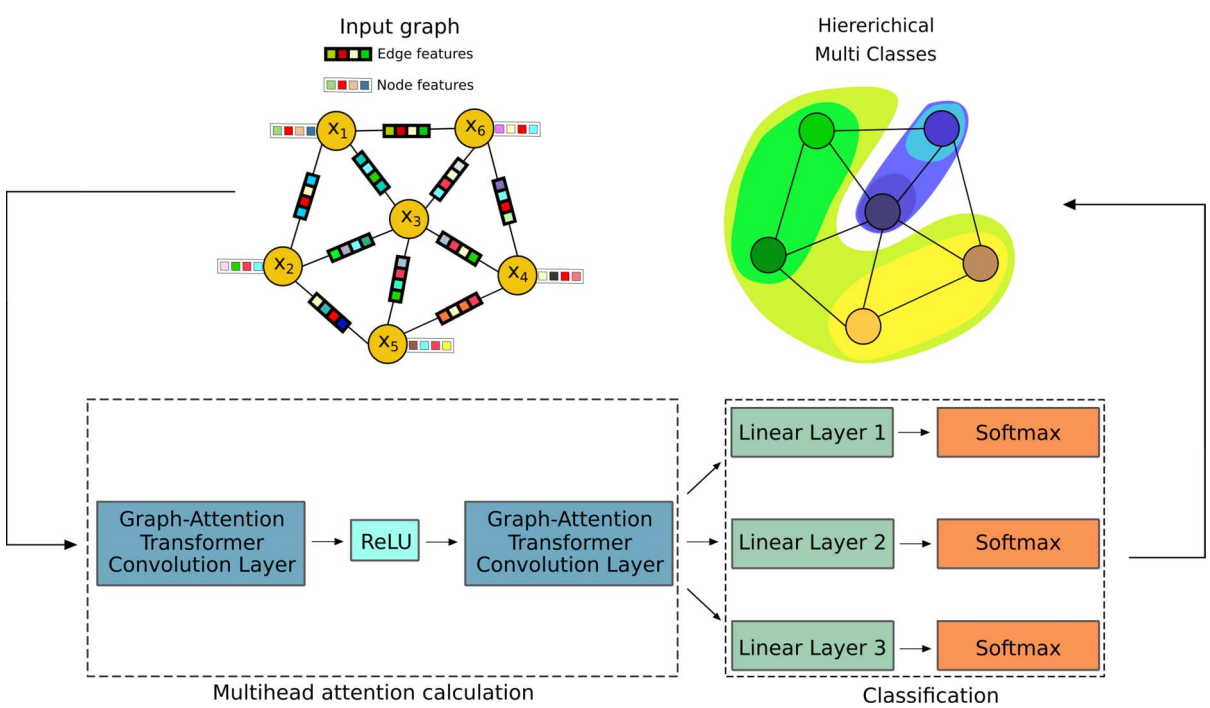


Fig. 7. Schematic of graph transformer architecture.

J. Comput. Civ. Eng., 2025, 39(6): 04025085

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

layer. The Graph Attention Layer updates each node's features, gathering information from its neighbors. This is weighted by adaptive learned coefficients, allowing the network to learn to select the most important neighbors following the formula:

$$\mathbf{x}_i' = \alpha_{i,i}\mathbf{\Phi}_s\mathbf{x}_i + \sum_{j\in\mathcal{N}(i)} \alpha_{i,j}\mathbf{\Phi}_t\mathbf{x}_j \tag{12}$$

The node's feature vector is defined by $\mathbf{x}_i'$, and this refinement is executed through data aggregation from adjacent nodes, employing aggregation weights defined by attention coefficients.

Considering a node i with its feature vector $\mathbf{x}_i'$. The node's intrinsic features transform a learnable weight matrix $\mathbf{\Phi}_s$ and are then scaled by its self-attention coefficient $\alpha_{i,i}$. Parallelly, the feature vectors $\mathbf{x}_j$ of each adjacent node j are altered through a distinct learnable weight matrix $\mathbf{\Phi}_t$ and subsequently scaled by the attention coefficient $\alpha_{i,j}$. The new feature vector $\mathbf{x}_i'$ results from the joint sum of these modified features, including the node and all its neighbors as follows:

$$\alpha_{i,j} = \frac{\exp(\mathbf{a}^\top \mathrm{LeakyReLU}(\mathbf{\Phi}_s\mathbf{x}_i + \mathbf{\Phi}_t\mathbf{x}_j + \mathbf{\Phi}_e\mathbf{e}_{i,j}))}{\sum_{k\in\mathcal{N}(i)\cup\{i\}} \exp(\mathbf{a}^\top \mathrm{LeakyReLU}(\mathbf{\Phi}_s\mathbf{x}_i + \mathbf{\Phi}_t\mathbf{x}_k + \mathbf{\Phi}_e\mathbf{e}_{i,k}]))} \tag{13}$$

The degree of influence that every neighbor, including the node itself, has on the updated feature vector is determined mostly by the attention coefficients.

Using the properties of both nodes, the attention coefficient $\alpha_{i,j}$ is inferred for a pair of nodes, i and j. The weight matrices $\mathbf{\Phi}_s$ and $\mathbf{\Phi}_t$ of node i and its neighbor j initially alter its attributes. These modified features are combined and passed through a LeakyReLU activation function to add nonlinearity. Furthermore, the edge characteristics between nodes i and j are considered by integrating an edge-specific term $\mathbf{\Phi}_e\mathbf{e}_{i,k}$. This combination is translated into a scalar value by a trainable vector $\mathbf{a}$, which is then exponential. Normalizing these exponential values across all the node i neighbors (including i) generates the final attention coefficient, ensuring a cumulative sum of 1.

The output of the second Graph Attention v2 layer is fed into a multiclass hierarchical classification module, which is designed to predict the class labels for each line in the drawing. This module consists of three parallel linear layers, each followed by a softmax activation function, to produce probabilistic outputs for each individual class. The classes in this context represent the hierarchical categories to which the lines can belong. These are derived from the layer summary of the CAD drawing and annotated with semantic labels. These labels denote the functional role of each element within the drawing. The hierarchy definition is explained in the Labels subsection, definition of extraction and hierarchy.

The primary classification task involves predicting whether a given line corresponds to one of the over-represented labels in the data set or a less common category. The second stage involves intermediate categorization, where lines are distinguished between general categories and more specific representations. The third and final stage represents each line's precise and detailed classification, providing a fine-grained categorization that distinguishes between subtly different features within the specific categories.

The weights for each classification task are shared across the hierarchical structure, enabling the model to leverage common patterns and features that are relevant across different classes. This shared-weight approach allows the model to capture class relationships and improve its overall performance. By doing so, the model can effectively identify the most relevant class labels for each line, which is essential for achieving accurate node classification and graph understanding. The hierarchical classification approach also helps to mitigate the issue of class imbalance, where certain classes may be underrepresented in specific subsets of the data set.

Experimental Setup

We present the experimental setup of our approach, which was evaluated using two distinct data sets: a proprietary data set comprising technical drawings from the Technical University of Munich (TUM) and the publicly available FloorplanCAD data set (Fan et al. 2021).

A side-by-side comparison of the TUM data set and FloorplanCAD, presented in Table 2, reveals key differences. TUM drawings exhibit significantly more lines per document on average (more than 13,000 lines versus 2,000 lines per patch in FloorplanCAD) and include 42 distinct fine-grained classes, many of which are unique to university campus designs (e.g., specialized installation and building connection). Conversely, FloorplanCAD is divided into 10,000 patches, each containing 2,000 lines. These two data sets thus represent distinct challenges: TUM emphasizes large-scale, single-document vector data, whereas FloorplanCAD focuses on panoptic segmentation tasks across many smaller patches.

The proprietary data set from the TUM consists of a comprehensive collection of floor plans for 10 different buildings on the university's main campus, each with multiple floors and diverging construction years. The data set includes a total of 76 floor plans, representing a diverse range of floor layouts and building structures. The floor plans are in SVG format and contain detailed information about the location of rooms, corridors, staircases, and other features.

The data set covers various building types and floor layouts, including basements, ground floors, and upper floors. Each building has a unique set of floor plans, with some buildings having as few as three or four floors and others having up to six or seven floors. The data set includes floor plans for buildings with different architectural styles and layouts, such as buildings with a central staircase, buildings with multiple wings, and buildings with a mix of open spaces and private offices.

TUM Data Set

The internal data set is obtained from CAD files that are exported in PDF format and undergo our conversion process to SVG format. This process ensures that the method works for each possible geometric file and does not require only the vector format directly from CAD drawing software. In the preprocessing phase, we filter out the text from the technical drawings to ensure that the learning is based only on geometrical information. The TUM Data Set consists of 76 Scalable Vector Graphics (SVG) files with sample

Table 2. Data set overview

Data set	Number of drawings/patches	Avg. lines per drawing/patch	Number of classes	Label type
FloorplanCAD	10,000+ patches	1–2,000	36	Panoptic symbols + line segmentation
TUM (Proprietary)	76 drawings	~13,500	42	Fine-grained building layers/annotations

J. Comput. Civ. Eng., 2025, 39(6): 04025085

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

images shown in Fig. 8. The minimum number of lines observed in a single drawing is 1,000, whereas the mean number of lines is approximately 13,500. Furthermore, the maximum number of lines present in a single drawing reaches 34,000. The architectural drawings comprise a data set of building information from various structures on the main campus of the TUM. The drawings are organized according to the floor plan of each building, providing a representation of the architectural features. Specifically, the drawings include structural elements, such as walls, columns, and beams, and hatching patterns to indicate wall textures and materials. Additionally, the drawings feature representations of architectural elements, including door and window placements, room layouts, and dimensional measurements of the components.

Labels Extraction and Hierarchy Definition

The graphical representation is annotated with semantic labels derived from the layer summary of the CAD drawing. These labels denote the functional role of each element within the drawing. The original German labels and their English translations are presented in Table 3, which also outlines the rule-based postprocessing applied to standardize the semantic labels.

The layer structure used in the CAD drawing is based on the Bavarian Procurement Manual for Professional Services (VHF Bayern), specifically section VI.4.2.H, which defines the standard for layer organization (Bayern 2018).

To mitigate the issue of class imbalance, where certain classes may be underrepresented in specific subsets of the data set, we aggregate layer classifications that occur in fewer than 33% of the total drawings in the class "Others." This strategy enables the model to learn a more generalized representation rather than overfitting to specific drawing types.

The label distribution in the data set exhibits significant class imbalance, with certain categories, such as "grid" and "others," being disproportionately represented. To address this issue, we adopt a multiclass hierarchical learning approach. The primary classification task involves determining whether a given line belongs to major categories such as "grid," "others," or one of the remaining specific labels in the data set.

The second class of labels provides an intermediate categorization of each line in the drawing, as shown in Table 3, where we distinguish between two primary categories: annotation lines and effective architectural elements. The annotation lines refer to lines that provide supplementary information or annotations to the drawing, such as dimension lines, leader lines, or text annotations. In contrast, architectural elements comprise lines representing actual physical components of the building or structure, including walls, doors, windows, and other architectural features. In the second stage, we group hierarchically based on semantic similarity, categorizing related elements into higher-level classes. For instance, we aggregate "door," "window," and "opening" into a single higher-level class denoted as "Windows and openings."

Ultimately, the third and final class represents each element's precise and detailed classification, providing a fine-grained categorization that distinguishes between subtly different architectural features, as shown in Table 3 through both numerical identifiers and descriptive labels.

During the vector graphics conversion process, inaccuracies or discrepancies can emerge in the data set, particularly in the numerical values associated with dimension lines and symbolic representations represented in Fig. 6. To mitigate this issue, we account for potential gaps in the calculation of connectivity rules governing the graph generation process, thereby ensuring the fidelity of the resulting data set.

FloorplanCAD Data Set

The flooplanCAD data set (Fan et al. 2021) constitutes a large-scale, real-world CAD drawing data set comprising more than 10,000 floorplans, encompassing a diverse range of residential and commercial buildings. The primary objective of this data set is panoptic symbol detection, which involves not only detecting countable objects such as furniture, windows, and doors in CAD drawings but also establishing the semantics of uncountable elements, including walls.

The data set is presented as patches, which are partial segments of larger technical drawings. Each patch provides a localized view of the original drawing.

One notable limitation of the FloorplanCAD data set is the absence of labels for all types of lines in the drawing. To address this limitation, we adopt the hierarchical grouping strategy based on semantic similarity, analogous to the approach employed in the TUM data set. The first class involves distinguishing between the symbol's lines and others. The second class is again the supergroup of elements having similar semantic definitions, and the third class is the precise classification of the symbol.

Results

Comparison with Existing Methods on FloorplanCAD Data Set

To evaluate the effectiveness of our proposed method, VectorGraphNET, we compared its performance against several state-of-the-art neural networks on the FloorplanCAD data set. The comparison metrics include F1-score and weighted F1-score (wF1), as shown in Table 4.

VectorGraphNET achieves a slightly lower F1-score compared to methods such as SymPoint (86.8) but significantly outperforms others in terms of weighted F1-score (wF1), achieving a score of 89.0. The wF1 metric is particularly crucial in this context because it accounts for class imbalance by weighting each class's F1-score through its prevalence in the data set. This highlights VectorGraphNET's robustness and effectiveness in handling complex architectural drawings with imbalanced class distributions.

Hierarchical Multilabel Approach Analysis

To evaluate the effectiveness of the proposed hierarchical multilabel learning approach, we performed a comparative analysis against a nonhierarchical single-label prediction baseline. The results presented in Table 5 include classwise metrics, such as Precision, Recall, and F1-score, for both approaches. Notably, the hierarchical method consistently outperforms the baseline, with the relative improvement in F1-score captured by a dedicated ΔF1 metric. This improvement highlights the benefit of leveraging hierarchical relationships in the data for more accurate and robust predictions.

This table contrasts the classification metrics (Precision, Recall, and F1-score) for each class in the FloorplanCAD data set under a nonhierarchical, single-label classification approach versus the proposed hierarchical multilabel method. The $\Delta F1$ column shows how much the hierarchical approach improves (or occasionally slightly reduces) F1 scores.

Notably, classes that initially scored zero under the nonhierarchical approach, such as "Revolving Door" and "Rolling Door," now achieve F1-scores of 0.89 and 0.91, respectively, demonstrating the hierarchical method's effectiveness for minority classes. More common classes, including "Single Door" (F1 rising from

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

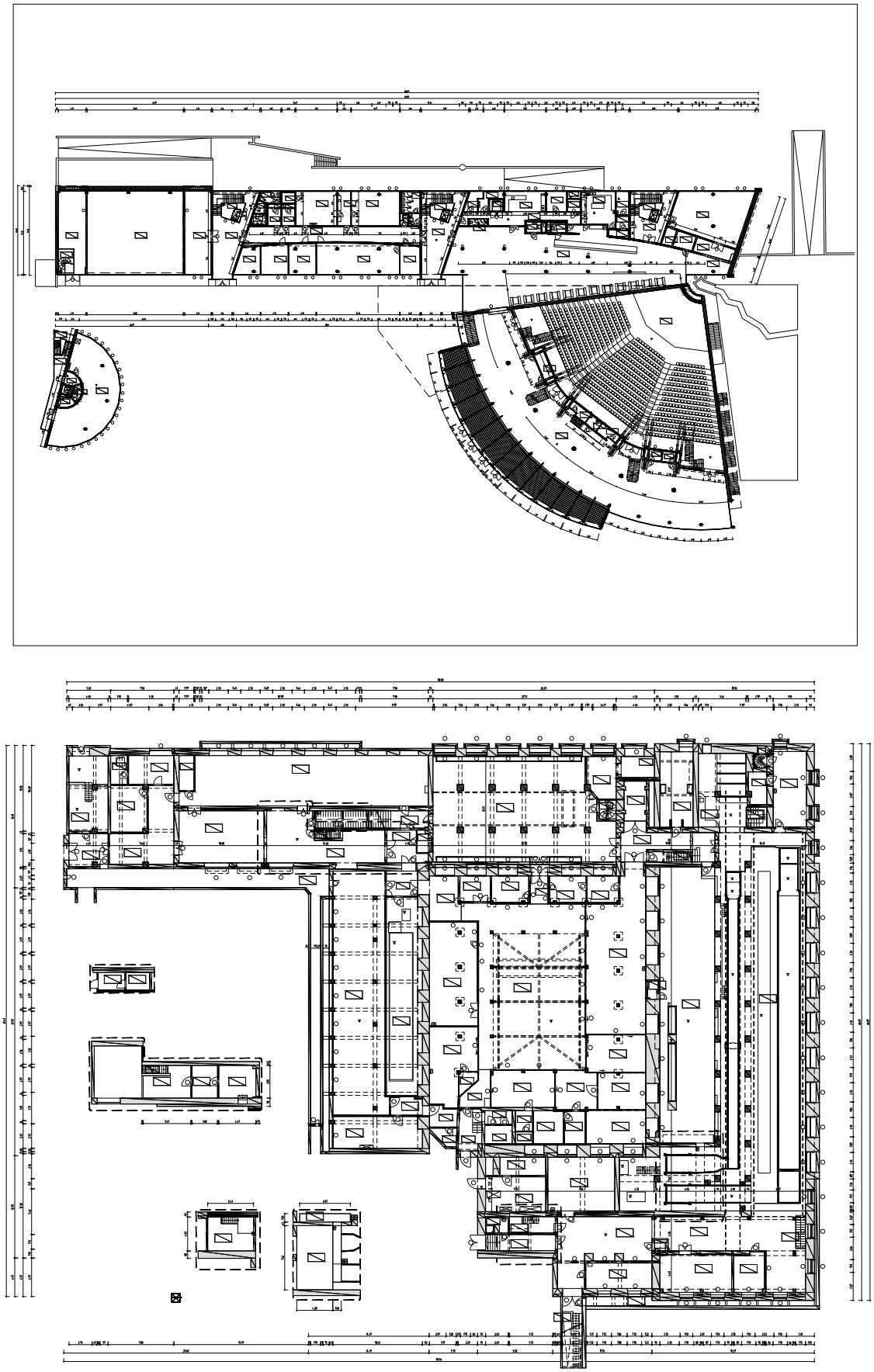


Fig. 8. Example of drawings in TUM data set.

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

Table 3. Annotations and their corresponding classes in TUM data set

Class 1	Class 2	Class 3	Percentage	Absolute	Name	Translation	Intermediate categorization
2	11	1	0.4049	1,074,982	A_00_RASTER	Grid	Annotations
1	0	13	0.0878	233,065	A_03_BEMASS	Dimension	Annotations
1	0	14	0.0003	714	A_03_HOEHENKOTE	Height dimension	Annotations
1	0	18	0.0030	7,848	A_08_UMRISS	Outline	Annotations
1	0	19	0.0006	1,701	A_08_UMRISS_VERDECKT	Hidden outline	Annotations
1	0	30	0.0203	54,008	A_19_BESTAND_SCHR	Existing building hatch	Annotations
1	0	31	0.0020	5,183	A_19_GEBAEUDEANSCHLUSS	Building connection	Annotations
1	0	37	0.0141	37,417	A_301_ADTRAUM	Room contour	Annotations
1	0	38	0.0018	4,805	A_301_ADTRAUM_BS	Room description box	Annotations
1	0	39	0.0041	10,883	A_401_BAUTEILTABELLE	Component table	Annotations
1	0	40	0.0005	1,198	A_431_MASSENELEMENT_ABZUG	Mass element deduction	Annotations
1	0	41	0.0010	2,736	A_60_SCHNITTLINIE	Section line	Annotations
1	0	42	0.0018	4,835	A_61_UEBERSICHT_DETAILS	Overview details	Annotations
1	7	21	0.0031	8,213	A_09_TRAGDECKE	Load-bearing ceiling	Ceiling
1	7	22	0.0008	2,016	A_09_TRAGENDE_BAUTEILE_2D	Load-bearing ceiling 2D	Ceiling
1	7	23	0.0110	29,209	A_10_UNTERZUG	Soffit	Ceiling
1	7	24	0.0028	7,459	A_11_LEICHTDECKE	Nonload-bearing ceiling	Ceiling
1	7	25	0.0004	978	A_11_LEICHTDECKE_BS	Nonload-bearing ceiling base	Ceiling
1	3	9	0.0180	47,730	A_014_STUETZE	Column	Columns Foundations
1	3	10	0.0159	42,298	A_015_FUNDAMENT	Foundation	Columns Foundations
1	4	15	0.0013	3,451	A_05_FASSADE	Façade	Façade
1	10	34	0.0126	33,417	A_26_MOEBEL_EINBAU	Furniture installation	Installations
1	10	35	0.0011	2,906	A_27_EINBAU_BESOND	Special installation	Installations
1	10	36	0.0013	3,537	A_28_EINBAU_SANIT	Sanitary installation	Installations
1	9	32	0.0002	474	A_22_AUSSPAR_DECKE	Roof joist	Joist
1	9	33	0.0003	695	A_23_AUSSPAR_BODEN	Floor joist	Joist
1	8	26	0.0004	955	A_12_DACH	Roof	Roof
1	8	27	0.0001	176	A_121_DACHELEMENT	Roof element	Roof
1	8	28	0.0001	257	A_122_DACHKONSTRUKTION	Roof construction	Roof
1	8	29	0.0000	119	A_123_DACHBELAG	Roofing	Roof
1	6	20	0.0028	7,340	A_09_3D_SONDER	Special constructions	Special
1	5	16	0.0019	5,017	A_06_TREPPE	External staircase	Stairs
1	5	17	0.0056	14,762	A_06_TREPPE_2D	Internal staircase	Stairs
0	0	0	0.1862	494,421	Others	Others	Undefined
1	1	2	0.0839	222,782	A_01_TRAGWAND	Load-bearing wall	Walls
1	1	11	0.0307	81,404	A_02_LEICHTWAND	Nonload-bearing wall	Walls
1	1	12	0.0002	662	A_021_TRENNWAND	Partition wall	Walls
1	2	3	0.0293	77,727	A_011_FENSTER	Window	Windows and openings
1	2	4	0.0016	4,200	A_012_OEFFNUNG	Opening	Windows and openings
1	2	5	0.0100	26,636	A_013_TUER	Door	Windows and openings
1	2	6	0.0038	10,126	A_013_TUER_BS	Door base	Windows and openings
1	2	7	0.0306	81,312	A_013_TUER_FENSTER_KOMBI	Door-window combination	Windows and openings
1	2	8	0.0020	5,421	A_013_TUER_OBERHALB	Upper door	Windows and openings

Table 4. Comparison of F1 and weighted F1 (wF1) scores on the FloorplanCAD data set

Method	PanCADNet	CADTransformer	GAT-CADNet	PointT‡	SymPoint	VectorGraphNET
F1	80.6	82.2	85.0	83.2	86.8	**79.4**
wF1	79.8	80.1	82.3	80.7	85.5	**89.0**

Note: Bold entries indicate either the best performance in a column or the overall (average) accuracy of each method.

0.77 to 0.87) and "Double Door" (0.85 to 0.90), also benefit. Although a few categories, such as "Sliding Door" and "Blind Window," show slight dips (−0.01 and −0.02), these decreases are negligible relative to the substantial gains elsewhere.

Overall, the macro average F1-score increases from 0.67 to 0.79. In contrast, the weighted average F1-score rises from 0.86 to 0.88, confirming that the hierarchical approach addresses class imbalance and maintains strong performance for majority classes. Thus, by leveraging hierarchical relationships, the proposed method yields a more balanced and accurate segmentation model for architectural drawings.

Despite the comparatively lower overall F1 score reported for VectorGraphNET in certain instances, it is essential to note that the weighted F1 (wF1) better reflects real-world performance in this context. Although a single summary metric such as the F1-score treats each class equally, many classes in architectural drawings occur infrequently and are thus prone to being overshadowed by more prevalent classes. In contrast, the wF1 metric integrates each class's contribution proportional to its frequency, thereby illustrating VectorGraphNET's robustness and adaptability in dealing with heavily imbalanced distributions. Consequently, even though the model's raw F1-score may not match the highest

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

Table 5. Classification report for FloorplanCAD data set. Comparison of Precision, Recall, and F1-score with (*Hierarch.*) and without (*Nonhierarch.*) hierarchical labels. The final column (ΔF1) represents the difference in F1-scores (Hierarchical minus Nonhierarchical)

		Nonhierarch.			Hierarch.			
Object	Class	Precision	Recall	F1	Precision	Recall	F1	ΔF1
Single door	0	0.80	0.74	0.77	0.85	0.90	**0.87**	+0.10
Double door	1	0.89	0.81	0.85	0.93	0.88	**0.90**	+0.05
Sliding door	2	0.92	0.86	0.88	0.91	0.84	**0.87**	−0.01
Folding door	3	0.97	0.45	0.62	0.86	0.81	**0.83**	+0.21
Revolving door	4	0.00	0.00	0.00	0.89	0.88	**0.89**	+0.89
Rolling door	5	0.00	0.00	0.00	0.92	0.91	**0.91**	+0.91
window	6	0.70	0.61	0.65	0.74	0.80	**0.77**	+0.12
Bay window	7	0.74	0.04	0.08	0.92	0.91	**0.91**	+0.83
Blind window	8	0.78	0.61	0.69	0.68	0.66	**0.67**	−0.02
Opening symbol	9	0.43	0.07	0.12	0.32	0.13	**0.19**	+0.07
Sofa	10	0.71	0.66	0.68	0.82	0.75	**0.78**	+0.10
Bed	11	0.88	0.91	0.90	0.91	0.90	**0.91**	+0.01
Chair	12	0.78	0.57	0.66	0.71	0.72	**0.71**	+0.05
Table	13	0.81	0.44	0.57	0.80	0.52	**0.63**	+0.06
TV cabinet	14	0.81	0.80	0.80	0.88	0.85	**0.86**	+0.06
Wardrobe	15	0.90	0.90	0.90	0.94	0.90	**0.92**	+0.02
Cabinet	16	0.81	0.67	0.73	0.84	0.69	**0.76**	+0.03
Gas stove	17	0.96	0.83	0.89	0.96	0.89	**0.93**	+0.04
Sink	18	0.87	0.81	0.84	0.89	0.84	**0.86**	+0.02
Refrigerator	19	0.81	0.67	0.73	0.91	0.73	**0.81**	+0.08
Air-conditioner	20	0.71	0.59	0.64	0.75	0.61	**0.67**	+0.03
Bath	21	0.62	0.40	0.49	0.69	0.53	**0.60**	+0.11
Bathtub	22	0.65	0.63	0.64	0.85	0.54	**0.66**	+0.02
Washing machine	23	0.90	0.82	0.86	0.93	0.86	**0.89**	+0.03
Urinal	24	0.95	0.93	0.94	0.96	0.94	**0.95**	+0.01
Squat toilet	25	0.81	0.93	0.86	0.87	0.93	**0.89**	+0.03
Toilet	26	0.87	0.89	0.88	0.88	0.93	**0.90**	+0.02
Stairs	27	0.88	0.75	0.81	0.88	0.78	**0.83**	+0.02
Elevator	28	0.82	0.85	0.84	0.86	0.87	**0.86**	+0.02
Escalator	29	0.57	0.55	0.56	0.69	0.53	**0.60**	+0.04
Row chairs	30	0.91	0.96	0.93	0.93	0.93	**0.93**	+0.00
Parking spot	31	0.93	0.86	0.90	0.92	0.87	**0.90**	+0.00
Wall	32	0.70	0.82	0.76	0.73	0.87	**0.79**	+0.03
Curtain wall	33	0.64	0.41	0.50	0.72	0.44	**0.55**	+0.05
Railing	34	0.39	0.37	0.38	0.45	0.37	**0.41**	+0.03
Other	35	0.90	0.91	0.90	0.91	0.92	**0.92**	+0.02
Accuracy			**0.86**			**0.89**		—
Macro avg		P = 0.74, R = 0.64, F1 = 0.67			P = 0.82, R = 0.76, F1 = 0.79			+0.12
Weighted avg		P = 0.86, R = 0.86, F1 = 0.86			P = 0.89, R = 0.89, F1 = 0.88			+0.02

Note: Bold entries indicate either the best performance in a column or the overall (average) accuracy of each method.

figures reported by other methods, the stronger wF1 underscores its reliability in capturing majority and minority elements within complex architectural floor plans.

Results on Technical University of Munich Data Set

The proprietary data set was utilized for training and testing, with a random split of 62 drawings for training and 14 for testing. The classification results, presented in Table 6, demonstrate that the proposed approach achieves an accuracy of 0.97, along with a macro average F1-score of 0.82 and a weighted average F1-score of 0.97. Notably, these results are reported at the most granular level of detail.

A visual comparison of ground truth and model predictions is provided in Fig. 9. A detailed classification report for each class is presented in Table 6. Unlike the previously analyzed data set, the proprietary data set was fed to the neural network model in its entirety, without dividing it into patches. This preserved the global design context and contributed to more coherent predictions. Moreover, although the earlier data set emphasized symbol spotting—grouping most nonsymbol elements into a general class—the proprietary data set assigned distinct classes to a broader set of lines. This granularity enabled the segmentation process to produce clearer boundaries and improved suitability for general-purpose, line-level predictions.

Comparison with CADTransformer on the TUM Data Set

To assess the generalizability and scalability of VectorGraphNET, we additionally trained CADTransformer (Fan et al. 2022) on the TUM data set using its publicly available implementation and recommended training protocol for 500 epochs. Although CADTransformer surpassed 90% training accuracy, it did not achieve comparable results on TUM's validation and test sets, highlighting challenges in adapting to the complexity and scale of these large architectural drawings.

A key factor is CADTransformer's reliance on patch-based, pixel-oriented processing developed initially for FloorplanCAD. That data set focuses on repeated symbols in smaller patches,

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

Table 6. Classification report for TUM data set. Comparison of Precision, Recall, and F1-score between (*CADTransformer.*) and (*VectorGraphNET*)

	CADtransformer			VectorGraphNET		
Object	Precision	Recall	F1	Precision	Recall	F1
Others	0.52	0.71	0.6	0.97	0.98	**0.97**
Grid	0.47	0.18	0.26	0.99	0.99	**0.99**
Load-bearing wall	0.55	0.77	0.64	0.96	0.95	**0.95**
Window	0.32	0.24	0.27	0.97	0.97	**0.97**
Opening	0	0	0	0.91	0.86	**0.88**
Door	0.26	0.35	0.3	0.91	0.91	**0.91**
Door base	0.52	0.64	0.57	0.93	0.88	**0.91**
Door-window combination	0.27	0.42	0.33	0.87	0.92	**0.9**
Upper door	0.02	0.02	0.02	0.88	0.86	**0.87**
Column	0.76	0.66	0.7	0.95	0.98	**0.96**
Foundation	0	0	0	1	0.96	**0.97**
Nonload-bearing wall	0.59	0.51	0.55	0.96	0.96	**0.96**
Partition wall	0.13	0.04	0.06	0.98	0.92	**0.95**
Dimension	0.88	0.93	0.91	0.99	0.98	**0.98**
Height dimension	0.27	0.27	0.27	0.82	0.77	**0.79**
FaÃ§ade	0.72	0.35	0.47	0.93	0.87	**0.9**
External staircase	0.38	0.14	0.21	0.84	0.87	**0.86**
Internal staircase	0.32	0.25	0.28	0.8	0.87	**0.83**
Outline	0.3	0.35	0.32	0.66	0.59	**0.62**
Hidden outline	0	0	0	1	0.98	**0.99**
Special constructions	0	0	0	0.91	0.87	**0.89**
Load-bearing ceiling	0.01	0.1	0.02	0.78	0.85	**0.81**
Load-bearing ceiling 2D	0	0	0	0.56	0.67	**0.61**
Soffit	0.52	0.13	0.21	0.85	0.91	**0.88**
Nonload-bearing ceiling	0.14	0.04	0.07	0.79	0.84	**0.82**
Roof	0.19	0.24	0.21	0.99	0.98	**0.98**
Roof element	0	0	0	0.89	0.99	**0.93**
Roof construction	0	0	0	0	0	**0**
Existing building hatch	0.73	0.23	0.35	0	0	**0**
Building connection	0.23	0	0	0.98	0.99	**0.98**
Roof joist	0.09	0.01	0.02	0	0	**0**
Floor joist	0.53	0.38	0.44	0.49	0.62	**0.55**
Furniture installation	0	0	0	1	0.96	**0.98**
Special installation	0.44	0.37	0.4	1	1	**1**
Sanitary installation	0.31	0.17	0.22	0.96	0.93	**0.95**
Room contour	0.57	0.75	0.65	0.87	0.93	**0.9**
Component table	1	1	1	0.91	0.97	**0.94**
Mass element deduction	0.18	0.23	0.2	0.9	0.7	**0.79**
Section line	0.31	0.44	0.37	0.36	0.84	**0.5**
Overview details	0.52	0.57	0.54	0.98	0.99	**0.98**
Accuracy		**0.66**			**0.97**	
Macro avg		P = 0.30, R = 0.27, F1 = 0.26			P = 0.81, R = 0.83, F1 = 0.82	
Weighted avg		P = 0.65, R = 0.66, F1 = 0.64			P = 0.97, R = 0.97, F1 = 0.97	

Note: Bold entries indicate either the best performance in a column or the overall (average) accuracy of each method.

making the method highly effective for local symbol detection and segmentation. However, when extended to TUM's full-floor architectural plans, the resolution demands increase substantially. As a result, pixel-level approaches lose critical details, and subdividing an entire floorplan into patches undermines global context—both of which are crucial for accurate line-level classification in dense, large-scale technical drawings.

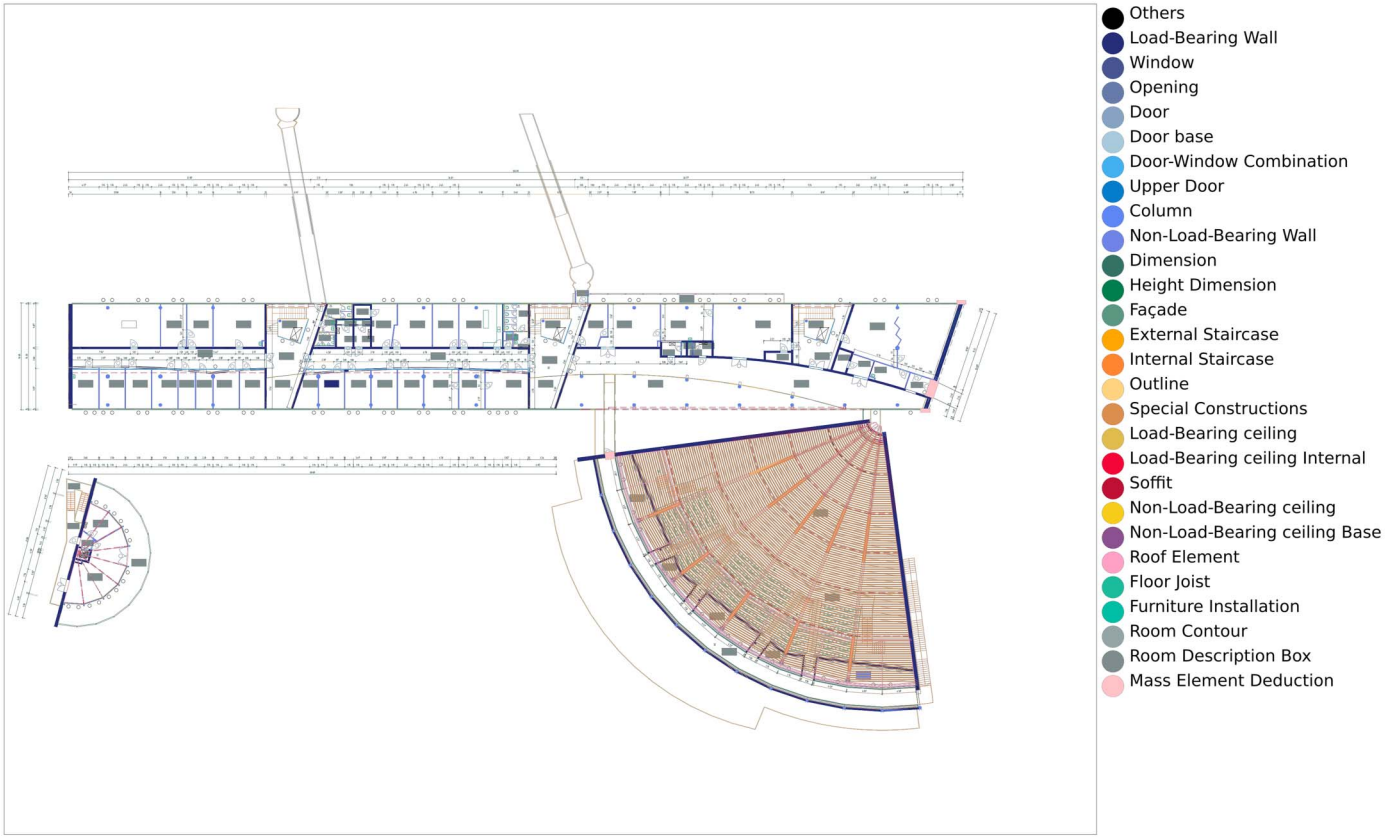
By contrast, VectorGraphNET operates on a vector-native graph representation. Rather than downsampling to pixel grids, our approach encodes geometric and relational information directly from the SVG data. This vector-based pipeline preserves fine details and global connectivity, allowing the network to capture intricate structural interdependencies. As evidenced by the substantial differences in macro and weighted F1 scores, VectorGraphNET is better equipped to parse and label floorplans where line density, overlapping components, and diverse geometric relationships introduce significant complexity.

Discussion

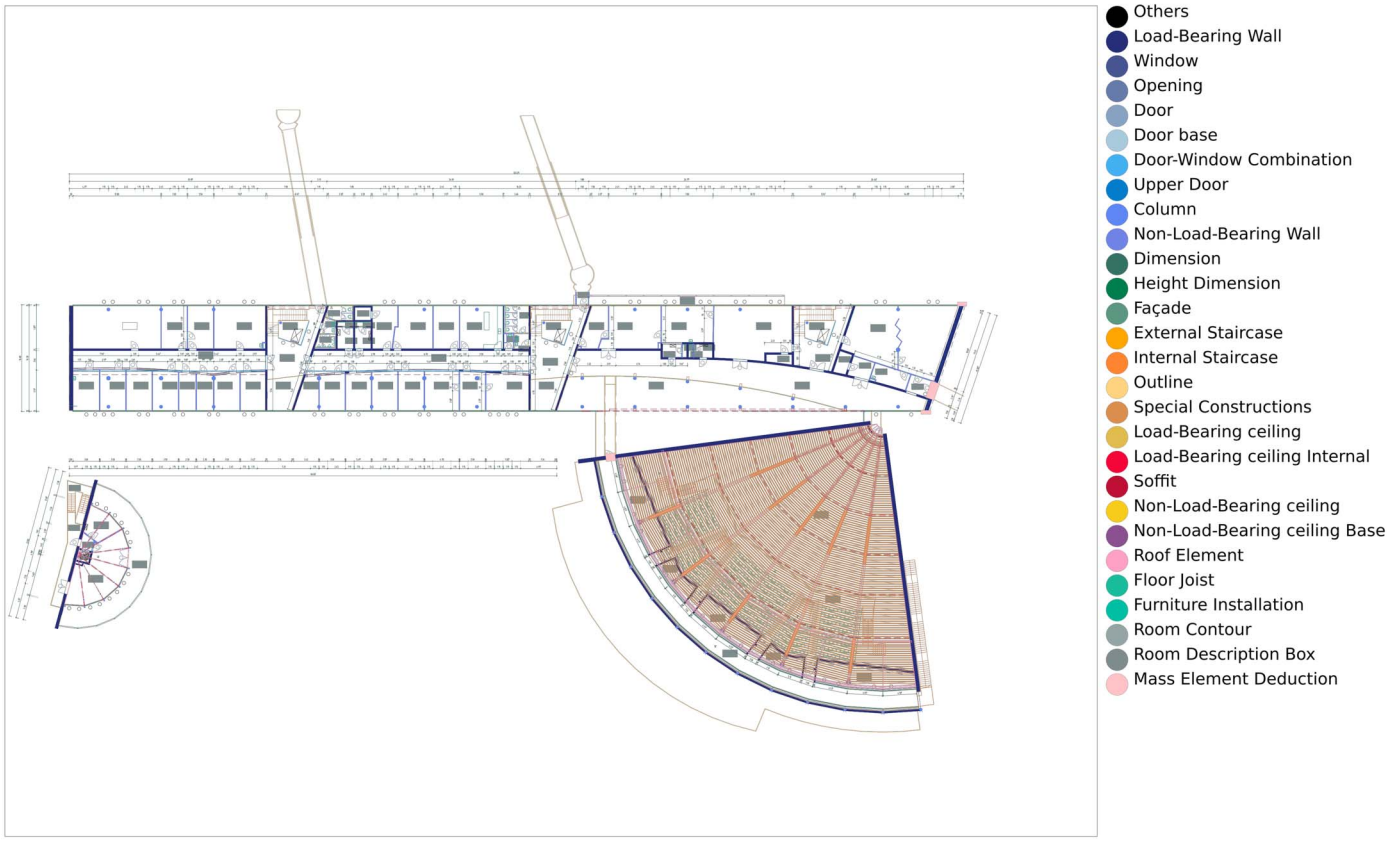
We evaluate the performance of our segmentation approach on the FloorplanCAD data set, comparing it to other state-of-the-art neural networks. Although VectorGraphNET achieves a competitive F1 score of 79.4, slightly lower than the top-performing SymPoint method (86.8), it significantly outperforms all other techniques, including SymPoint, in terms of weighted F1 (wF1) with a score of 89.0. Notably, the wF1 metric is this task's most critical evaluation criterion because it accounts for class imbalance and accurately represents the model's performance. This superior performance in wF1 highlights our method's effectiveness in addressing class imbalance and demonstrates its robustness in handling complex architectural drawings.

Our neural network architecture is lightweight, with only 1.3 million weights compared to existing implementations such as PanCADNet (>42 M), CADTransformer (>65 M), PointT‡Cluster

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.



(a)



(b)

Fig. 9. Segmentation results comparison between ground truth and predicted labels on TUM data set: (a) GT; and (b) model segmentation.

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

(31 M), and SymPoint (35 M)—all featuring neural networks with more than 30 million weights. This reduction in complexity is mainly due to our efficient approach of precomputing an extensive graph that, although computationally expensive upfront, enables efficient processing on a lighter GPU. By performing this graph computation on the CPU, we can leverage parallelization at a lower cost than that incurred when parallelizing the neural network computation on multiple GPUs. Furthermore, this precomputation step allows us to operate on a simplified representation of the data assuming that the postprocessing of every vector file is correct and all vectors are in plain representation. As a result, our approach can be trained on a single NVIDIA RTX 8000, featuring 48 GB of GDDR6 memory and 672 GB/s of memory bandwidth, in contrast to SymPoint, which requires an impressive computational infrastructure of 8 NVIDIA A100 GPUs with a combined 384 GB of HBM2 memory and 12,440 GB/s of memory bandwidth. This highlights the efficiency and scalability of our proposed architecture. Our model demonstrates efficient training, requiring only 1 min per epoch on the NVIDIA RTX 8000 GPU, and achieves a wF1 score of 0.85 after 50 epochs, with peak performance reached at 500 epochs.

Notably, our approach differs from existing methods, relying solely on geometrical vector information without considering CAD primitives or layers. This design choice makes our approach more generic and widely applicable to any drawing in vector format regardless of the vector's origin or structure. In contrast, methods that rely on CAD-specific information are inherently limited to CAD-specific applications, which restricts their usability. By avoiding CAD-specific dependencies, our approach is more robust to variations in CAD software, file formats, and user workflows and can adapt more easily to changes in the field.

Practical Considerations and Limitations

Although the proposed method demonstrates strong performance on detailed vector representations, it is important to acknowledge that rasterized or scanned drawings remain present in AEC projects, particularly in historical archives, legacy documentation, and ongoing projects involving renovations or existing structures. Although contemporary workflows increasingly favor vector-based CAD formats, many existing records and active projects still rely on rasterized scans. In such cases, a preliminary vectorization step is required to reconstruct geometric primitives from pixel-based representations. Automated vectorization from images is inherently susceptible to inaccuracies, particularly for older, low-resolution scans, which can introduce errors that degrade the reliability of downstream line-level semantic segmentation.

Purely raster-based approaches leveraging CNNs or traditional computer vision techniques do not rely on vector data but typically require higher computational costs and exhibit limitations when processing complex line structures, such as overlapping lines. A potential solution lies in hybrid methodologies that initially extract high-confidence vector primitives while incorporating pixel-based processing for ambiguous regions. Because the industry has transitioned toward CAD-based workflows and vector-based formats, native vector-based approaches are becoming applicable for large-scale deployment. Nevertheless, these challenges underscore the need for continued research to improve the interoperability between scanned (raster) and native CAD (vector) formats, ensuring robust and adaptable solutions for AEC applications.

Conclusion

In this study, we presented an efficient approach to semantic segmentation of architectural drawings using a hierarchical multilabel definition. Our method, VectorGraphNET, achieves state-of-the-art performance on the FloorplanCAD data set, outperforming existing methods in weighted F1 score. Notably, our approach is remarkably lightweight, with only 1.3 million weights, making it more efficient and scalable than existing implementations. Our results demonstrate the effectiveness of our approach in addressing class imbalance and handling complex architectural drawings. Using a hierarchical multilabel definition allows our model to capture the underlying patterns in the data, leading to improved classification performance. Furthermore, our approach is more generic and widely applicable to any drawing in vector format regardless of its origin or structure. The efficiency and scalability of our proposed architecture make it an attractive solution for real-world applications. In conclusion, our work presents a significant contribution to the field of semantic segmentation of architectural drawings. Our approach offers a more efficient, scalable, and widely applicable solution for real-world applications, and we believe that it has the potential to be used in a variety of fields, including architecture, engineering, and construction.

Data Availability Statement

Some or all data, models, or code that support the findings of this study are available from the corresponding author upon reasonable request.

Acknowledgments

The presented research has received funding from the Bayerisches Verbundforschungsprogramm (BayVFP) des Freistaates Bayern – Förderlinie "Digitalisierung, grant number DIK0336/01. The authors thank TUM Central Services for providing the CAD drawings used in this project.

Author Contributions

Andrea Carrara: Conceptualization; Methodology; Software; Writing – original draft; Writing – review and editing. Stavros Nousias: Supervision; Writing – review and editing. André Borrmann: Funding acquisition; Project administration; Supervision; Writing – review and editing.

References

Ahmed, S., M. Liwicki, M. Weber, and A. R. Dengel. 2011. "Improved automatic analysis of architectural floor plans." In *Proc., 2011 Int. Conf. on Document Analysis and Recognition*, 864–869. New York: IEEE.

Bayern, V. 2018. "Vi.4.2.h layerstruktur." Accessed September 27, 2024. https://www.stmb.bayern.de/assets/stmi/buw/bauthemen/vergabeundvertragswesen/vhf/vi-4-2_layerliste_2018.pdf.

Borrmann, A., M. König, C. Koch, and J. Beetz. 2018. "Building information modeling: Why? What? How?" In *Building information modeling*. Cham, Switzerland: Springer.

Brody, S., U. Alon, and E. Yahav. 2021. "How attentive are graph attention networks?" Preprint, submitted May 30, 2021. https://arxiv.org/abs/2105.14491.

Carlier, A., M. Danelljan, A. Alahi, and R. Timofte. 2020. "DeepSVG: A hierarchical generative network for vector graphics animation." Preprint, submitted July 22, 2020. https://arxiv.org/abs/2007.11301.

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.

Carrara, A., S. Nousias, and A. Borrmann. 2024. "Employing graph neural networks for construction drawing content recognition." In *Proc., i3CE 2024: 2024 ASCE Int. Conf. on Computing in Civil Engineering*. Reston, VA: ASCE.

Cheng, B., I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar. 2022. "Masked-attention mask transformer for universal image segmentation." Preprint, submitted December 2, 2021. https://arxiv.org/abs/2112.01527.

Dodge, S., J. Xu, and B. Stenger. 2017. "Parsing floor plan images." In *Proc., 2017 15th IAPR Int. Conf. on Machine Vision Applications (MVA)*, 358–361. New York: IEEE.

Dong, S., W. Wang, W. Li, and K. Zou. 2021. "Vectorization of floor plans based on EdgeGAN." *Information* 12 (5): 206. https://doi.org/10.3390/info12050206.

Fan, Z., T. Chen, P. Wang, and Z. Wang. 2022. "CADTransformer: Panoptic symbol spotting transformer for CAD drawings." In *Proc., 2022 IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 10976–10986. New York: IEEE.

Fan, Z., L. Zhu, H. Li, X. Chen, S. Zhu, and P. Tan. 2021. "FloorPlanCAD: A large-scale CAD drawing dataset for panoptic symbol spotting." Preprint, submitted May 15, 2021. https://arxiv.org/abs/2105.07147.

Girshick, R. 2015. "Fast R-CNN." In *Proc., 2015 IEEE Int. Conf. on Computer Vision (ICCV)*, 1440–1448. New York: IEEE.

ISO. 2008. *Document management—Portable document format—Part 1: Pdf 1.7*. ISO 32000. Geneva: ISO.

Kalervo, A., J. Ylioinas, M. Häikiö, A. Karhu, and J. Kannala. 2019. "CubiCasa5K: A dataset and an improved multi-task model for floorplan image analysis." Preprint, submitted April 3, 2019. https://arxiv.org/abs/1904.01920.

Lazaro-Aleman, W., F. Manrique-Galdos, C. Ramirez-Valdivia, C. Raymundo-Ibañez, and J. M. Moguerza. 2020. "Digital transformation model for the reduction of time taken for document management with a technology adoption approach for construction SMEs." In *Proc., 2020 9th Int. Conf. on Industrial Technology and Management (ICITM)*, 1–5. New York: IEEE.

Liu, C., J. Wu, P. Kohli, and Y. Furukawa. 2017. "Raster-to-vector: Revisiting floorplan transformation." In *Proc., 2017 IEEE Int. Conf. on Computer Vision (ICCV)*, 2214–2222. New York: IEEE.

Liu, W., T. Yang, Y. Wang, Q. Yu, and L. Zhang. 2024. "Symbol as points: Panoptic symbol spotting via point-based representation." Preprint, submitted January 19, 2024. https://arxiv.org/abs/2401.10556.

Lv, X., S. Zhao, X. Yu, and B. Zhao. 2021. "Residential floor plan recognition and reconstruction." In *Proc., 2021 IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 16712–16721. New York: IEEE.

Macé, S., H. Locteau, E. Valveny, and S. Tabbone. 2010. "A system to detect rooms in architectural floor plan images." In *Proc., Int. Workshop on Document Analysis Systems*. New York: Association for Computing Machinery.

Moreno-García, C. F., E. Elyan, and C. Jayne. 2019. "New trends on digitisation of complex engineering drawings." *Neural Comput. Appl.* 31 (Jun): 1695–1712. https://doi.org/10.1007/s00521-018-3583-1.

Rasmussen, M. H., M. Lefrançois, P. Pauwels, C. A. Hviid, and J. Karlshøj. 2019. "Managing interrelated project information in AEC knowledge graphs." *Autom. Constr.* 108 (Dec): 102956. https://doi.org/10.1016/j.autcon.2019.102956.

Simonsen, C. P., F. M. Thiesson, M. P. Philipsen, and T. B. Moeslund. 2021. "Generalizing floor plans using graph neural networks." In *Proc., 2021 IEEE Int. Conf. on Image Processing (ICIP)*, 654–658. New York: IEEE.

Surikov, I. Y., M. A. Nakhatovich, S. Y. Belyaev, and D. A. Savchuk. 2020. "Floor plan recognition and vectorization using combination UNet, faster-RCNN, statistical component analysis and Ramer-Douglas-Peucker." In *Computing science, communication and security*, edited by N. Chaubey, S. Parikh, and K. Amin, 16–28. Singapore: Springer.

Tariq, J., and S. S. S. Gardezi. 2023. "Study the delays and conflicts for construction projects and their mutual relationship: A review." *Ain Shams Eng. J.* 14 (1): 101815. https://doi.org/10.1016/j.asej.2022.101815.

W3C SVG Working Group. 2023. "Scalable vector graphics (SVG) 2.0 specification (Editor's draft, March 8, 2023)." Accessed September 26, 2024. https://svgwg.org/svg2-draft/.

Wang, W., S. Dong, K. Zou, and W.-S. Li. 2021. "Room classification in floor plan recognition." In *Proc., 4th Int. Conf. on Advances in Image Processing, ICAIP '20*, 48–54. New York: Association for Computing Machinery. https://doi.org/10.1145/3441250.3441265.

Wu, W. 2024. "Architectural floorplan recognition via iterative semantic segmentation networks." In *Proc., 2023 7th Int. Conf. on Computer Science and Artificial Intelligence, CSAI '23*, 282–287. New York: Association for Computing Machinery. https://doi.org/10.1145/3638584.3638636.

Yamasaki, T., J. Zhang, and Y. Takada. 2018. "Apartment structure estimation using fully convolutional networks and graph model." In *Proc., 2018 ACM Workshop on Multimedia for Real Estate Tech, RETech'18*, 6–1. New York: Association for Computing Machinery. https://doi.org/10.1145/3210499.3210528.

Zeng, Z., X. Li, Y. K. Yu, and C.-W. Fu. 2019. "Deep floor plan recognition using a multi-task network with room-boundary-guided attention." In *Proc., 2019 IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 9095–9103. New York: IEEE.

Zhang, Y., Y. He, S. Zhu, and X. Di. 2020. "The direction-aware, learnable, additive kernels and the adversarial network for deep floor plan recognition." Preprint, submitted January 30, 2020. https://arxiv.org/abs/2001.11194.

Zhao, H., L. Jiang, J. Jia, P. Torr, and V. Koltun. 2021. "Point transformer." Preprint, submitted December 16, 2020. https://arxiv.org/abs/2012.09164.

Zheng, Z., J. Li, L. Zhu, H. Li, F. Petzold, and P. Tan. 2022. "GAT-CADNet: Graph attention network for panoptic symbol spotting in CAD drawings." Preprint, submitted January 3, 2020. https://arxiv.org/abs/2201.00625.

This work is made available under the terms of the Creative Commons Attribution 4.0 International license.